

Структура та програмна модель комп'ютера



©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Філософськи-практичний вступ

- *Програмування* – різновид інженерії¹.
- Інженерна справа включає вирішення задач в рамках обмежень накладених як законами природи, так і наявними ресурсами та вимогами замовників.
- Зазвичай, існує декілька способів вирішити задачу, поміж яких варто прагнути знайти найкращий.
- Різні вимоги та обмеження практично завжди конфліктують між собою – маємо *інженерний конфлікт*, вирішення якого – знаходження *інженерного компромісу*.

Результатом цієї лекції буде каркас того, що ми вивчатимемо далі. Своєрідна розмальовка, яку потім заповнюватимемо. Будь ласка, будьте уважними!

¹Хоча представники сфери ІТ іноді це й заперечують...

Два епіграфи

- "As far as the areas that I've been personally involved in – mostly high-speed networking – the best hardware/software tradeoffs have been mostly just judgment calls by very experienced folks. What I've noticed is that a *LOT* of "lessons learned" decades ago are still being re-learned by later generations, over and over again. We as an industry don't seem to have been doing a very good job of passing on the "secret sauce" to subsequent generations [in many cases, even *within* the same companies!]", Rob Warnock writing in comp.arch on 16 Jul 2001.*
- "[A] noteworthy characteristic of this manual is that it doesn't always tell the truth. When certain concepts of TeX are introduced informally, general rules will be stated; afterwards you will find that the rules aren't strictly true. [...] The author feels that this technique of deliberate lying will actually make it easier for you to learn the ideas. Once you understand a simple but false rule, it will not be hard to supplement that rule with its exceptions.", Donald Knuth, "The TeXBook"².*

²Натрапив на цю цитату тут: ["Make things simpler than possible"](#) – цікавий аналіз освітніх спрощень. ◀ ▶ ≡ 🔍 ↺

- Хто не бачив класичної картинки на тему, з чого складається комп'ютер?

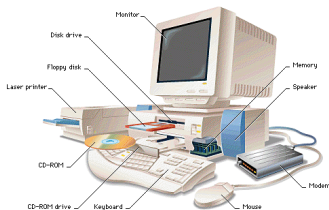


Fig 1.1 The hardware parts of a computer system

- На ній не зупинятимемося.
- Також зараз не будемо зупинятися на детальнішому вивченні окремих його частин, їх ролі та історії – поки важливим є лише загальне уявлення про структуру та взаємодію компонент.
- ◇ Детальніше ці питання розглядатимуться в майбутньому³.

³Іронія – історія тієї чи іншої сфери, зазвичай, розглядається на початку підручника чи курсу. Тобто, тоді, коли вона ще не цікава переважній більшості користувачів та не може принести користі від аналізу в ретроспективі.

Елементи комп'ютера

Для нас важливим буде усвідомлення існування наступних компонент:

- Центральний процесор⁴ – Central processing unit, CPU.
- Оперативна пам'ять⁵ – Random Access Memory, RAM.
- Засоби вводу-виводу – Input/Output, I/O.

Поміж засобів вводу-виводу, виділимо – заради простоти, відмовившись від повноти розгляду чи ортогональності:

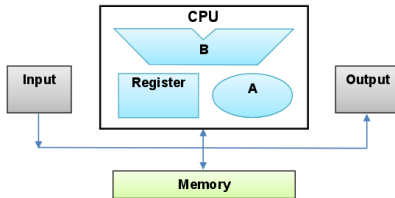
- Зовнішня енергонезалежна, (вторинна), часто – дискова, пам'ять⁶.
- Засоби інтерфейсу користувача та взаємодії із зовнішнім світом – клавіатура, мишка, відеокарта із дисплеєм, мережеві картки та модеми всіх технологій тощо.
- ◇ В цій категорії – надзвичайно різноманітні пристрої, тому мають різні вимоги до реалізації (до прикладу: час реакції, пропускну здатність), використовуються свої трюки та компроміси для реалізації.

⁴Мікропроцесор – це інтегральна схема, яка реалізовує CPU. Зараз – майже синонім CPU.

⁵Іноді називають первинною чи основною пам'яттю.

⁶Всі розуміють, про що мова, але через зоопарк технологій, дати означення важко.

- CPU, RAM та пристрої вводу-виводу з'єднані та обмінюються інформацією за допомогою **шин** (bus).
- Що таке шини, як вони функціонують, навіщо нам DMA⁷, переривання і т.д. – розглядатимемо в подальшому. Поки вважатимемо їх абстрактними каналами взаємодії – передачі інформації, повідомлень та команд.
- Враховуючи різноманітність вимог та задач, шини дуже різні – реалізують різні комбінації компромісів.



⁷І що це за літерки, BTW.

Різновиди шин

Грубо⁸ виділимо наступні шини:

- шину пам'яті⁹,
- "внутрішні"¹⁰ шини вводу-виводу, такі як PCIe – PCI-express¹¹ та SATA¹²,
- зовнішні шини – для підключення різноманітної периферії, такі як USB, FireWire, legacy COM-порти тощо.
- ◇ Зазвичай, зовнішні шини, такі як USB чи й SATA, реалізуються через спеціалізовані пристрої (хаби), що з'єднані із рештою комп'ютера PCIe чи іншою "внутрішньою" шиною.
- Набір схем, які реалізують взаємодію всіх цих компонент, називають чіпсетом (chipset).

⁸Нагадую, поки ми не прагнемо до повноти розгляду чи ортогональності – нам слід зрозуміти загальну схему. На практиці – в реальних комп'ютерах, часто все не зовсім так, іноді – зовсім не так.

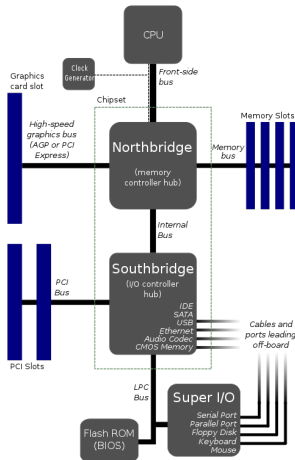
⁹Критичну щодо швидкодії, це накладає серйозні обмеження на неї.

¹⁰Це не термін! Лише умовна назва для наших потреб.

¹¹Та PCI/AGP/ISA – швидкісні, як на свій час, хоча й повільніші за шини пам'яті і з іншими задачами.

¹²А ось NVMe – протокол поверх PCIe, TCP, RDMA тощо.

Материнська плата



- Зображення вище вже застаріло – через жорсткі вимоги до продуктивності, сучасні процесори більшість Northbridge "втягнули" в себе. Чіпсет, що виконує залишки його функцій та функції Southbridge, часто називають Platform Controller Hub (PCH). Однак, концептуально воно залишається правильним.

Взаємодія компонент комп'ютера – доступ до RAM

- RAM – Random Access Memory, оперативна пам'ять.
- Кожен байт RAM має свій номер¹³.
- Власне, елементарну адресовану комірку пам'яті – таку, яка має адресу і до якої можна безпосередньо звернутися, називають байтом¹⁴.
- ◇ Існували різні розміри байтів, історично зафіксувалося значення 8 біт¹⁵. Воно достатньо зручне.
- Процесор звертається до пам'яті за допомогою **шини пам'яті**.
- ◇ Для цього він повідомляє пам'ять про операцію, яку він хоче здійснити – читання чи запис та номер байта, який його цікавить.

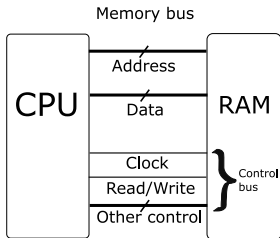
¹³Бувають нюанси. Наприклад, банки пам'яті – свого роду паралельні її світи. Звичною справою є те, що нумерація не неперервна. Фізично, пам'ять часто передаватиме багато байт за раз. Не всі процесори дозволяють незалежно звертатися до кожного байту, або такі звертання, хоча й можливі, є повільними. Подробиці розглядатимемо в майбутньому.

¹⁴Історично це була кількість біт, достатня для кодування одного символу.

¹⁵Ймовірно, завдяки популярності IBM System/360, яка використовувала цей розмір, потім підхоплений революцією мікропроцесорів – 8-бітовими 8080 і т.д.

Шина пам'яті

- Шина пам'яті – шина, якою процесор та інші частини комп'ютера звертаються до RAM.
- Вона складається із декількох частин, по одній для кожної задачі, їх теж називають шинами:
 - ◇ шина адреси,
 - ◇ шина даних,
 - ◇ шина керування.



- Візуалізація¹⁶: <https://yarynakorduba.github.io/ram-viz/>.
- ◇ За посиланням – **емулятор** роботи із пам'яттю. Це поняття нам не раз ще знадобиться.

¹⁶Розроблена Яриною Кордубою, нашою випускницею.

Читання та запис у RAM

- Для виконання операції читання:
 - ◇ Процесор встановлює на шині адресу комірки пам'яті, яка його цікавить¹⁷.
 - ◇ Отримавши шиною керувальний сигнал, пам'ять виставляє на шині даних вміст відповідного байта.
 - ◇ Процесор читає його із шини даних.
- Для запису:
 - ◇ Процесор виставляє адресу даних, в яку слід записати, на шині адреси.
 - ◇ Дані, які потрібно записати – на шині даних.
 - ◇ Після – віддає команду пам'яті шиною керування.
 - ◇ Пам'ять записує дані із шини даних у байт з номером, переданим шиною адрес.

¹⁷Зазвичай, сучасні CPU побайтово, чи й машинними словами, з пам'яттю не обмінюються – за раз передається помітно більша порція даних. Тут для нас це не принципово.

Взаємодія компонент комп'ютера – input/output (I/O)

- Шини вводу-виводу¹⁸, зазвичай, організовані трішки по іншому.
- ◇ Вони дуже різноманітні – деякі розглядатимемо в майбутньому.
- До такої шини часто під'єднано декілька пристроїв, тому, перш ніж почати обмін, слід домовитися, котра пара пристроїв зараз спілкуватиметься.
- Часто на шині є один або кілька¹⁹ головних пристроїв (master, manager) та якась кількість підлеглих (slave, worker).
- Master ініціює обмін та керує ним. Зокрема, вирішує, із котрим пристроєм відбуватиметься взаємодія.
- Шини вводу-виводу, так чи по іншому, під'єднані до шини пам'яті – щоб дані із периферійних пристроїв могли потрапляти в RAM і назад.

¹⁸Периферійні шини, як їх ще називають.

¹⁹І тоді їм ще доводиться між собою домовлятися – так званий арбітраж шини.

Команди I/O

- Пристрої вводу-виводу значно різноманітніші, ніж RAM.
- ◇ Тому взаємодія із ними включає не тільки обмін даними, але й обмін **командами**²⁰. Наприклад: "перейди в режим вимірювання прискорення", "встанови вказану роздільну здатність", "передай три останні виміри", "засни".

²⁰Фактично, пам'ять теж розуміє команди – "запиши", "прочитай" у прикладі вище, значно більше таких команд у реальних сучасних блоків пам'яті, однак, їх мало, в порівнянні із командами I/O-пристроїв.

Взаємодія компонент комп'ютера – переривання

- Периферійні пристрої, зазвичай, багато повільніші за CPU²¹.
- ◇ Крім того, вони функціують асинхронно відносно CPU.
- ◇ Активне очікування (busy loop) є дуже неефективним.
- Тому часто використовуються **апаратні переривання**.
- За їх допомогою, підлеглий пристрій на шині, slave, може повідомити процесор, що у нього новини – прибули дані, завершено операцію тощо.
- Тоді процесор, за першої нагоди, може прореагувати, не відволікаючись решту часу.
- ◇ **IRQ – Interrupt request**, запит на переривання від периферії. Має свій номер, що визначає походження IRQ.
- ◇ **ISR – Interrupt service routine**, підпрограма, яка опрацьовує запит на переривання. Мають свої номери.
- ◇ Відповідність номера IRQ номеру ISR встановлює **контролер переривань**. Він також керує пріоритетом переривань.

²¹Так було не завжди, однак, CPU стають швидшими помітно швидше за інші пристрої

Взаємодія компонент комп'ютера – DMA

- Відволікати процесор лише щоб передати дані від пристрою в пам'ять чи на інший пристрій, може бути неефективним²².
- Тому використовується **Direct Memory Access – DMA**.
- DMA контролер – пристрій на шині, який, як і процесор, може здійснювати обмін даних нею.
- Процесор інструктує DMA контролер – коли прийдуть дані від такого-то пристрою (про що повідомить переривання) – класти їх туди. І займається своїми справами.
- В загальному випадку, DMA контролер може виконувати обмін RAM-I/O, I/O-RAM, RAM-RAM чи I/O-I/O.

²²Мова про код, який виконується на процесорі та дає команду: прочитати байт із RAM чи від периферії у так званий регістр – внутрішню пам'ять процесора, потім – записати байт із регістра у пам'ять чи передати шиною даних

Взаємодія компонент комп'ютера – memory mapped I/O

- Ввід-вивід процесор може здійснювати двома різними підходами.
- Перший – окремі машинні команди, призначені для вводу-виводу.
- Другий – периферійні пристрої "шпигують" (слухають) шину адреси RAM, і для певних адрес, вважають, що це звертання до них, а не до RAM (остання в цей час мовчить).
- Такий підхід називають memory-mapped I/O – ввід-вивід ініціюється звертанням до певних комірок пам'яті.
- ◇ Приклад: запис коду букви в байт номер B8000h для певної архітектури²³, призводить до того, що вона з'являється в лівому верхньому кутку екрану. Тобто, це звертання, насправді, перехоплюється відеоадаптером. Оперативної пам'яті, яка впізнала б цю адресу, немає²⁴
- Власне, за допомогою memory-mapped I/O, DMA може організовувати обмін даними між периферією, без залучення CPU – з його точки зору він лише керує обміном даними по шині пам'яті.

²³Мова про архітектуру IBM PC/AT – предка всіх сучасних не-Apple персональних комп'ютерів.

²⁴Є відеопам'ять – однак, це пам'ять іншого пристрою, відеокарти, яка доступна за допомогою шини вводу-виводу. Хоча, буває по всякому – див. деякі вбудовані відеокарти.

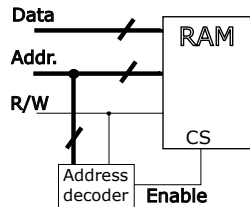
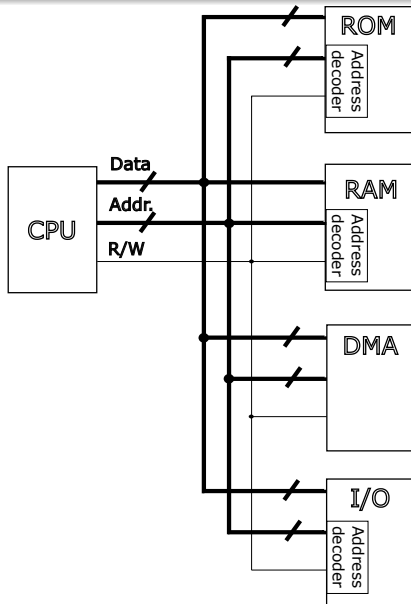
Kernel mode та user mode

- Щоб мати можливість створювати безпечні та надійні операційні системи та інше програмне забезпечення, потрібно, щоб CPU підтримував розмежування привілеїв.
- Хоча реалізації сильно відрізняються, загальна ідея наступна. Є мінімум два режими роботи CPU: привілейований (supervisor або kernel mode) та користувацький (user mode).
 - ◇ В привілейованому режимі можна виконувати будь-які операції.
 - ◇ В користувацькому заборонене все, що може не дати акуратно написаній ОС убезпечити себе та інші процеси від некоректної поведінки одного із них.

ISA – Instruction Set Architecture

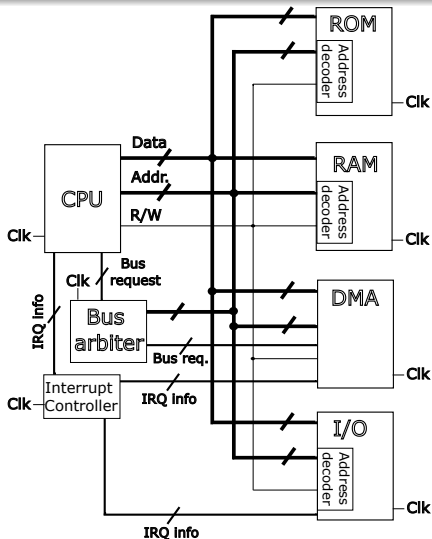
- Існують різні варіанти CPU. Видиму програмістам архітектуру процесора називають Instruction Set Architecture – ISA. Детальніше ми це поняття будемо розглядати в майбутньому.
- ◇ Більшість персональних комп'ютерів та високопродуктивних серверів використовують ISA з іменем x86.
- ◇ ISA ARM (різних варіантів) використовується в мобільних пристроях – смартфонах, планшетах тощо, мікроконтролерах та сучасних комп'ютерах Apple із M1 та новішими процесорами.
- ◇ AVR8 – ISA мікроконтролерів, використовуваних в більшості плат Arduino.
- ◇ Z80 – ISA, дуже популярна в перших персональних комп'ютерах (кінець 1970-х/початок 1980-х), зараз використовується у спеціалізованих пристроях.
- ◇◇ Z80 можна назвати далеким, побічним, родичем x86.

Різноманітні пристрої на шині пам'яті



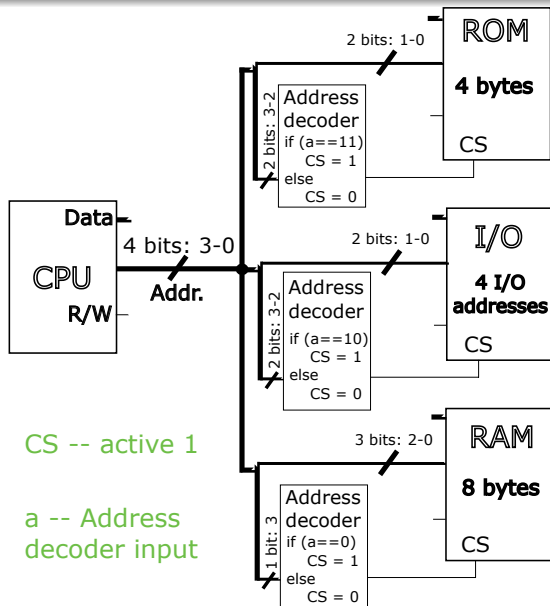
- Схема максимально спрощена, звичайно.

Різноманітні пристрої на шині пам'яті – повніша схема



- Див. також ще цю, дуже гарно ілюстровану відповідь: "[What is Memory Mapping in Microprocessor based systems?](#)" від Balajee Seshadri.

Приклад комп'ютерної системи із 4-бітовою шиною адрес – 1



CS -- active 1

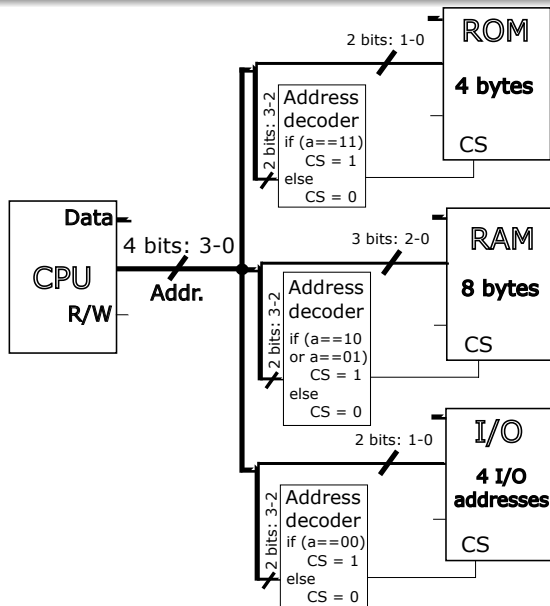
a -- Address decoder input

Карта адресного простору:

1111	
1110	
1101	ROM
1100	
1011	
1010	
1001	I/O
1000	
0111	
0110	
0101	
0100	
0011	
0010	
0001	
0000	

- З'єднання шини даних та R/W не показано, щоб не відволікати.

Приклад комп'ютерної системи із 4-бітовою шиною адрес – 2

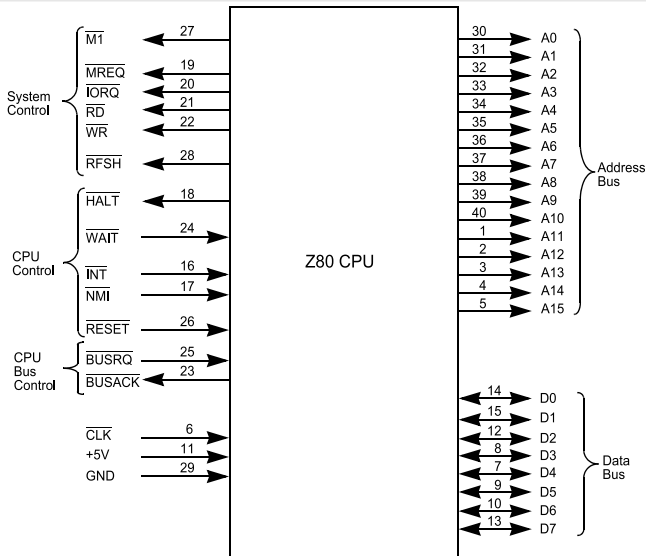


Карта адресно-
го простору:

1111	
1110	
1101	ROM
1100	
1011	
1010	
1001	
1000	
0111	RAM
0110	
0101	
0100	
0011	
0010	
0001	I/O
0000	

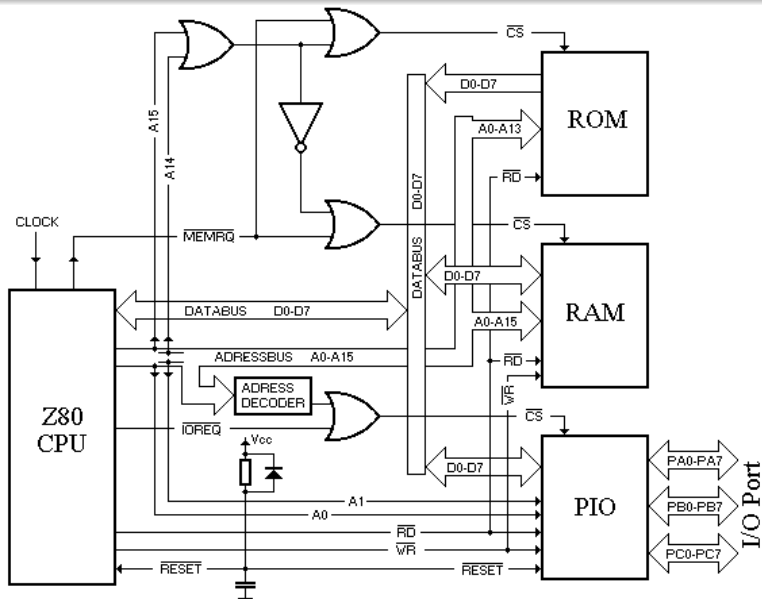
- Дещо складніший випадок.

Z80 – приклад дуже простого процесора



- Схему взято з офіційної документації,
<https://www.zilog.com/docs/z80/um0080.pdf>

Комп'ютер на базі Z80



Види пристроїв

- Пристрої вводу-виводу можна розділити на два різновиди: блокові та символьні.
- **Блоковий пристрій** підтримує ввід-вивід лише блоками фіксованого розміру. Доступ до блоків може відбуватися довільно, за їх номером. Диски, сховище (storage), є блоковими пристроями.
- ◇ Типовим розміром блоку сучасних HDD є 4Кб, раніше було популярним значення 512 байт. В SSD – розміри найрізноманітніші, але програмно часто емулюється²⁵ значення для HDD.
- **Символьний пристрій** підтримує обмін посимвольно, пошук місця для вводу-виводу неможливий.
- ◇ Символьними є клавіатури, мишки, інші USB-пристрої, мережеві карти тощо.
- Через різноманітність пристроїв і для оптимізації, конкретні операційні системи виділяють й інші класи пристроїв.

²⁵ Детальніше це поняття розглядається нижче, якщо коротко.

- Розмова про апаратне забезпечення змушувала вживати багато нових слів – обійти це поки неможливо.
- Однак, говорячи про програмне забезпечення, можна почати із знайомого вам та рухатися вниз по рівнях абстракції.
- Почнемо із тривіальної задачі: прочитати файл із послідовністю дійсних чисел та вивести їх середнє значення і дисперсію:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

- Будемо також вважати, що всі дані для обчислень ми хочемо завантажити у пам'ять – для ще якогось аналізу – обчислити дисперсію можна і без цього:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 = \dots = \left(\frac{1}{N} \sum_{i=1}^N x_i^2 \right) - \bar{x}^2$$

Мотив: для подальшого розгляду хотілося б мати привід читати в пам'ять невідомий наперед об'єм даних, а ускладнювати логіку коду не хотілося б.

- Програма на Python, яка це робить, може виглядати якось так:

```
import math

with open("data.txt", "r") as datafile:
    data = [float(x) for x in datafile.read().split()]
    average = sum(data)/len(data)

std_dev = math.sqrt(sum((x-average)**2 for x in data)/len(data))

print(average)
print(std_dev)
```

- Обробка помилок у цій програмі, як і у більшості нижче, рудиментарна, щоб не відволікати від суті, однак, все ж, присутня.

Аналіз коду

- ❶ Програма вважає, що дані знаходяться у файлі "data.txt".
- ❷ При тому, файл цей має знаходитися в **поточній директорії**.
- ❸ Перш ніж можна буде щось робити із файлом, слід повідомити операційній системі (**ОС**) про ваші наміри. Це – процес *відкриття файлу*. ОС при цьому перевіряє, чи такий файл існує та чи має дана програма права із ним працювати.
- ❹ Запит на відкриття файлу програма здійснює за допомогою функції `open()`.
- ❺ При цьому програма вибирає режим роботи із файлом. Тут – лише читання, про що свідчить аргумент "r".
- ❻ Після завершення роботи із файлом, його слід *закрити* – повідомити ОС про це та дати їй можливість "підбити підсумки" – оновити розмір файлу, дату його зміни, звільнити внутрішні структури даних із ним пов'язані, розпочати процес очищення буферів тощо.

- ❖ Для цього до файлового об'єкту Python можна застосувати метод `close()`. Однак, якщо в процесі роботи над файлом станеться виключення – до завершення процесу, що його відкрив, файл так і залишиться відкритим. Тому тут ми скористалися менеджером контексту `with ... as`, який автоматично викличе `close()`, не залежно від причин виходу з нього. Без менеджера контексту код виглядав би так:

```
datafile = open("data.txt", "r")
file_content = datafile.read()
.....
datafile.close()
```

- ❖ Для маніпуляції із файловим об'єктом ми користуємося таким засобом Python як *посилання* (reference). Менеджер контексту чи безпосередній виклик `open()` створює відповідний об'єкт, (в процесі відбувається звертання до ОС і все таке) та повертає нам "ниточку" до нього – спосіб із ним взаємодіяти.

- 9 Наступна задача – прочитати весь вміст файлу та зберегти його в пам'яті процесу.
- 10 Ми вважаємо, що дані – послідовність чисел, збережено там у вигляді *тексту*, а числа розділяються так званими пробільними символами²⁶ (whitespaces).
- 11 Для комп'ютера текстовий формат чисел незручний – слід виконати перетворення. В Python найзручніший тип дробових чисел, точніше, чисел із рухомою комою/крапкою називають float, в багатьох інших мовах – double.
- 12 Хорошою практикою програмування є виражати свої високорівневі наміри, а не детально описувати процес.
- 13 Враховуючи це все, читання повного вмісту файлу виглядатиме так:

```
data = [float(x) for x in datafile.read().split()]
```

Тут використано так звану генерацію списків чи list comprehension. Читати цей, дещо дивний, код можна так: "прочитати весь файл, розбити по пробільних символах, перетворити кожен елемент у float і результат помістити у список".

²⁶До них, крім власне пробілу, входять також табуляції, символ нового рядка (обидва під Windows, якщо ви розумієте, про що мова – якщо ні, нижче це теж розглядатиметься.)

- 14 Менш елегантний, ймовірно – повільніший, але більш безпосередній та звичний програмістам на багатьох інших мовах варіант виглядатиме так:

```
1 file_content = datafile.read()
2
3 data = []
4 for n in file_content.split():
5     data.append(float(n))
```

Поки що вручну поділ на елементи не розглядатимемо – все ще використано `split()`.

- 15 Тут, для списку `data`, як і для файлового об'єкту, створеного `open()`, Python бере на себе маніпуляцію пам'яттю. Як він це робить – одна із ключових тем цієї лекції.
- 16 Зауважте: при написанні програми ми вже неявно маємо на увазі, **що існує декілька різних видів пам'яті**: RAM та пам'ять для тривалого зберігання (HDD чи SSD), при тому, остання – повільніша.
- 17 Далі програма нарешті виконує арифметичні операції (над змінними в пам'яті у форматі `double`), для яких й була створена.

- 18 Знайшовши середнє значення та дисперсію, програма повинна якось продемонструвати результат. Для цього вона користується викликом функції `print()`.
- 19 `print()` виводить у так званий `stdout` – *стандартний вивід*.
 - ◇ Зразу варто згадати концепцію Unix: "все є файлом".
- 20 За замовчуванням `stdout` пов'язаний з консоллю – чорним²⁷ текстовим віконечком.
- 21 Нарешті, коли процес завершується, ОС повинна звільнити всі виділені для нього ресурси – пам'ять, файли тощо.

Хоча, сподіваюся, всі чи більшість цих етапів вам відомі та усвідомлені, подивитися на них під таким кутом важливо для подальшого розгляду.

²⁷ Колір залежить від вподобань користувача.

Процеси і програми

- **Процес**²⁸ – одна із ключових абстракцій комп'ютерного світу.
- Детальніше вона розглядатиметься в курсі ОС.
- Зараз лише домовимося, що процес – копія програми в RAM, яка виконується.
- Іншими словами, програма – пасивна колекція команд, а процес здійснює фактичне виконання цих команд.
- На процес зручно дивитися як на контейнер ресурсів, пов'язаних із екземпляром програми, що виконується, разом із, мінімум, одним потоком виконання. Поміж цих ресурсів – виділена пам'ять, відкриті файли, завантажені динамічні бібліотеки, поточна директорія²⁹ і т.д. і т.п.
- При завершенні процесу, сучасні операційні системи здатні звільнити більшість виділених йому ресурсів³⁰.

²⁸Н. Deitel у своїй книзі "Operation systems", 2nd ed., стверджує, що термін "процес" вперше було вжито операційною системою Multics.

²⁹Також називають "поточним шляхом" – це шлях у файлової системі, який використовується за замовчуванням для пошуку файлів, – якщо не вказано інший безпосередньо. В процесі виконання процес може змінювати свою поточну директорію.

³⁰Раніше бувало й по іншому. Наприклад, популярна колись MS-DOS часто не мала шансів самостійно звільнити виділену процесом так звану Extended чи Expanded пам'ять. Винятки сучасних ОС, часто пов'язані із ресурсами відлагодження та профілювання програм – всілякі апаратні та програмні лічильники. Див., наприклад, підсистему Event Tracing for Windows (ETW).

Трансляція

- Вище було розглянуто просту програму – інструкції комп'ютеру.
- Питання – хто саме її виконуватиме і як?
- Комп'ютер, напевне. Однак, *як відомо*³¹, він працює із нулями та одиничками, а наша програма – якийсь текст, навіть із звичними англійськими словами.
- Тобто, щоб виконати цю програму, нам знадобиться *транслятор*.
- Транслятор "перекладає" програму із однієї *мови програмування* на іншу.
- Результат трансляції може бути або переданий ще одному транслятору, або виконаний безпосередньо – якщо цільову мову трансляції розуміє апаратне забезпечення.

³¹Все ж, що це означає, розуміє помітно менша частка людей, ніж ті, що цю фразу чули.

Бінарний код

- Найпростіший приклад³² – запуск стандартної програми `ls` Unix-системи (Linux) чи `notepad.exe` – стандартного блокноту MS Windows.
- Обидві програми вже є в **бінарному коді**³³, зрозумілому процесору, для якого вони призначені.
- Тобто, після необхідних перетворень, призначених перетворити програму – файл чи файли на диску, у процес – код, що виконується, які здійснює операційна система, **код безпосередньо виконується центральним процесором**.
- Процесор транслює двійкові (машинні) команди у якісь свої внутрішні дії, які, в кінцевому результаті, дають бажаний користувачам результат.
- Детальніше ми бінарний код розглядатимемо в майбутньому. Якщо коротко, кожна команда, яку розуміє процесор, має свій двійковий код. Набір команд процесора називають системою команд – `instruction set architecture (ISA)`.

³²Для розгляду на обраному нами рівні абстракції та деталізації.

³³Детальніше, що це значить, вивчатимемо в подальшому.

Інтерпретовані мови

- Python демонструє інший підхід до трансляції.
- Заради однозначності, вважатимемо, що використовується транслятор CPython від <https://www.python.org>.
- Це – так званий *інтерпретатор*. Він безпосередньо виконує код програми на Python, рядок за рядком.
- Сам *інтерпретатор* CPython – бінарна програма, яка виконується на центральному процесорі.

Компілятори та асемблери

- Питання – звідки беруться двійкові програми? Їх створює інший різновид трансляторів – *компілятори*.
- Виділяють також спеціальний клас таких трансляторів – асемблери (Assembler), які транслюють із спеціальної (мнемонічної) форми представлення машинного коду (Assembly language) у, власне, машинний код.
- Важливим інструментом є також редактор зв'язків чи лінкер, який об'єднує окремі частини програми в єдиний виконавчий модуль. Детальніше його діяльність розглядається на практичних.

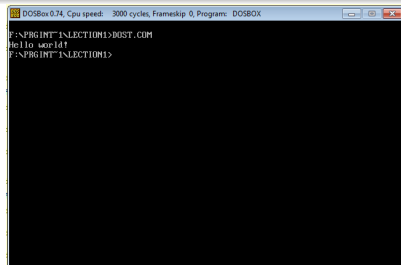
Приклад коду на асемблері

- Програмування в машинних кодах:

```
10110100 00001001 10111010
00001001 00000001 11001101
00100001 11001101 00100000
01001000 01100101 01101100
01101100 01101111 00100000
01110111 01101111 01110010
01101100 01100100 00100001
00100100
```

Або так:

```
B4 09 BA 09 01 CD 21 CD 20
48 65 6C 6C 6F 20 77 6F 72
6C 64 21 24
```



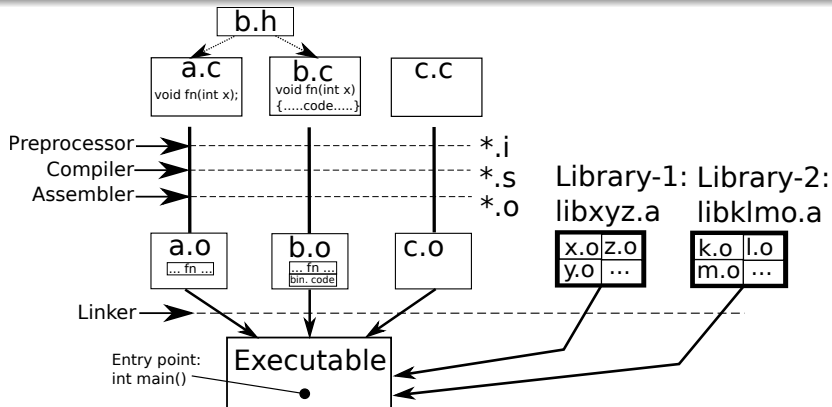
```
org      100h
use16
```

```
mov      ah,09h      ; B4 09
mov      dx,hello    ; BA 09 01
int      21h         ; CD 21
```

```
int      20h         ; CD 20
```

```
hello db      'Hello world!$'
; 48 65 6C 6C 6F 20 77 6F 72 6C
; 64 21 24
```

Роздільна компіляція



- Розширення – ті, що, за замовчуванням, використовує GCC.
- Проміжні файли, в нормі, не зберігаються. Щоб вони залишилися, слід скористатися ключем `-save-temps`.
- За замовчуванням, використовується AT&T стиль асемблера. Перекопати використовувати стиль Інтел: `-masm=intel`. Для кращого сприйняття варто також додати `-fverbose-asm`.
- Схема дуже спрощена, більш детальну, на прикладі Windows: ["A graphical depiction of the steps in building a C++ executable, enhanced for classic Win32"](#) та ["A graphical depiction of the steps in building a C++ executable, with XAML and packaging"](#).

Байт-код та віртуальна машина Java

- Програму на Java, зазвичай, компілюють.
- Однак, у так званий **байткод** (bytecode) – машинний код віртуальної машини Java (**JVM**)³⁴.
- Віртуальна машина Java цей код виконує, інтерпретуючи. Зроблено так заради переносимості³⁵.

³⁴Та інших віртуальних машин. На загаль, підходів до інтерпретації є два – абстрактні синтаксичні дерева та байткод. Перші зберігають більше семантики інтерпретованої мови, другий – ближчий до цільової машини, тому швидший. Детальніше про них та про проміжні варіанти, див. "[AST, Bytecode, and the Space In Between: An Exploration of Interpreter Design Tradeoffs](#)".

³⁵Слоганом Java є "Write once, run anywhere" – в ідеальному варіанті програма пишеться раз, компілюється раз, а виконується на віртуальній машині, яку, звичайно, треба написати для кожної платформи, але це робиться лише раз та й багато коду між платформами спільного.

Асемблер JVM

- ```
public class test_asm {
 public static void main(String args[]) {
 int n1 = 225, n2 = 115;
 n1 += n2;
 }
}
```

- Компілюємо:

```
javac test_asm.java && javap -v test_asm.class > test_asm.asm
```

- Отримуємо (фрагмент):

```
public static void main(java.lang.String[]);
descriptor: ([Ljava/lang/String;)V
flags: (0x0009) ACC_PUBLIC, ACC_STATIC
Code:
```

```
 stack=2, locals=3, args_size=1
 0: sipush 225
 3: istore_1
 4: bipush 115
 6: istore_2
 7: iload_1
 8: iload_2
 9: iadd
 10: istore_1
 11: return
```

LineNumberTable:

# Байт-код та асемблер PVM (Python) та WebAssembly

- Python:

```
>>> import dis
>>> dis.dis('a+=b')
1 0 LOAD_NAME 0 (a)
 2 LOAD_NAME 1 (b)
 4 INPLACE_ADD
 6 STORE_NAME 0 (a)
 8 LOAD_CONST 0 (None)
 10 RETURN_VALUE
```

- WebAssembly:

```
add(int, int):
 local.get 1
 local.get 0
 i32.add
 end_function
```

- Див. також есей про IR LLVM: ["LLVM Hello World"](#).

## Чимдалі в ліс...

- Крім CPython існують й інші реалізації цієї мови, наприклад Jython<sup>36</sup> та IronPython<sup>37</sup>.
- Перший із них – реалізація Python поверх віртуальної машини Java, другий – поверх "альтер-его" Java від Microsoft<sup>38</sup> – поверх Common Language Runtime (CLR)<sup>39</sup>, віртуальної машини .NET.
- Вони обоє дозволяють легко використовувати у коді на Python об'єкти, створені для відповідних віртуальних машин.
- Також у цих реалізаціях відсутній GIL – Global Interpreter Lock.

### Спільним для них є:

- Кожна із них існує і як інтерпретатор, і як компілятор.
- У варіанті інтерпретатора код на Python виконується байткодом, скажімо, Java, у відповідній віртуальній машині, який, у свою чергу, віртуальною машиною виконується на апаратному забезпеченні. Хоча, бувають нюанси – див. далі.
- У варіанті компіляції код *компілюється* в байткод.

<sup>36</sup>На жаль, проект майже заморожений – до Python3 так і не був оновлений, станом на 2022. Див. також <http://www.jython.org>.

<sup>37</sup>Тривалий час проект не переходив на Python3 та почав видаватися закинутим, але в 2022 було випущено версію для 3.4.0. Детальніше див. <http://ironpython.net>.

<sup>38</sup>Повчальною є історія спроб Microsoft завести собі свою Java та народження .NET.

<sup>39</sup>Точніше, поверх Dynamic Language Runtime, яка, у свою чергу – поверх CLR 

## Чимдалі в ліс...

- Ще одним прикладом є GraalVM<sup>40</sup> – віртуальна машина Java, реалізована на Java.
- На додачу до JVM мов, підтримує JavaScript (Node.js), Python, Ruby, R та має SDK для додавання нових.
- Вміє запускати код мовами програмування типу C чи C++, для яких, зазвичай, генерується нативний код<sup>41</sup>.
- ◇ Для цього потребує clang – транслює LLVM bitcode<sup>42</sup>, отриманий із **компіляції** коду C/C++, у байткод Java.
- Дозволяє комбінувати програми, написані різними мовами – аналог C calling conventions, за допомогою яких спілкується нативний код сучасних ОС.
- "PyXL – GPIO round-trip at 480ns" – процесор на FPGA, який безпосередньо виконує (байт)код Python.
- "Collapsing towers of interpreters" – приклад наукових пошуків покращення продуктивності таких систем.

<sup>40</sup> Детальніше див. <https://habr.com/ru/company/haulmont/blog/433432/> і, скажімо, <https://github.com/graalvm/graalpython>.

<sup>41</sup> Бінарний (об'єктний) код конкретної платформи.

<sup>42</sup> Проміжний результат компіляції clang-ом.

- Крім компіляції та інтерпретації, існує багато проміжних варіантів.
- Наприклад, багато інтерпретаторів (включаючи JVM) за першого<sup>43</sup> виконання можуть *компілювати* код у машинне представлення цільової платформи, і вже потім виконувати його – з тією ж ефективністю, що й "рідний", *нативний* (native) код платформи – **just-in-time compilation** (JIT).
- Сучасна JVM виконує ряд інших важливих трюків-оптимізацій, наприклад, компілює байт-код із врахуванням даних про те, як він виконувався.
- ◇ Див. також profile-guided optimization.
- PyPy<sup>44</sup> – реалізація Python, яка, на відміну від CPython, покладається на JIT<sup>45</sup>.

---

<sup>43</sup>Або і-того, залежно від ситуації.

<sup>44</sup>До 3.9 включно – повністю сумісна із "чистим" CPython, але станом на 2022, має труднощі із деякими засобами 3.10.

<sup>45</sup>Також, цікавий есей про використання JIT в "звичайному" Python: "[Decorator JITs – Python as a DSL](#)"    

# JIT – посилання

- Хороший цикл статей про JIT: "[A Deep Introduction to JIT Compilers: JITs are not very Just-in-time](#)", "[How JIT Compilers are Implemented and Fast: PyPy, LuaJIT, Graal and More](#)".
- Інший цикл: "[Adventures in JIT compilation: Part 1 – an interpreter](#)", "[Part 2 – an x64 JIT](#)", "[Part 3 – LLVM](#)", "[Part 4 - in Python](#)".
- "[Polynima - Practical Hybrid Recompile for Multithreaded Binaries](#)", про перетворення бінарників в IR і компіляцію наново, з новими вимогами.

# Оптимізація

- Компілятор часто виконує якісь перетворення програми, щоб зробити її швидшою. Подробиці обговорюватимуться на практичних.
- Поняття Undefined behavior існує, в значній мірі, щоб спростити оптимізацію<sup>46</sup>.
- Link-time optimization – назва технології оптимізації **цілої** програми – на противагу оптимізації кожного об'єктного файлу, без інформації про решту проекту.
- Profile-guided optimization – оптимізація із врахуванням **профілів**, які отримуються під час запусків програми на реалістичних даних. Вони безпосередньо показують, де є динамічні гарячі місця коду.
- ◇ Без цієї інформації компілятор мусить покладатися на припущення.

---

<sup>46</sup>Але див. "Defining All Undefined Behavior and Leveraging Compiler Transformation APIs"

# Оптимізація – посилання

- Серія статей з прикладами оптимізацій, від одного з розробників LLVM:
  - ◇ "Compiler Optimization in a Language you Can Understand"
  - ◇ "Compiler Optimization in a Language you Can Understand – The Appendix"
  - ◇ "Introduction to Scalar Evolution with Iteration-based Intuition"
  - ◇ "Loop to Constant Computation"
- "Common Misconceptions about Compilers"

# Інтерпретація vs компіляція – 1

- Приклад із Jython/IronPython ілюструє важливу ідею.
- Ми звикли вживати слова "інтерпретована мова програмування" чи "компільована мова програмування", однак, вибір компілювати чи інтерпретувати робить розробник транслятора.
- Мови, які ми звикли сприймати як компільовані, мають свої інтерпретатори (включаючи C<sup>47</sup>, C++<sup>48</sup> та FORTRAN).
- Для інтерпретованих мов часто існують компілятори. Приклад: Nuitka, компілятор Python (2.7, 3.13 на 2025) в C++ код (поки що – сумісний із C99, без засобів, власне, C++), написаний на Python: <https://nuitka.net/>.
- Цікавий приклад – езотерична мова Befunge, із двомірним кодом: <https://esolangs.org/wiki/Befunge>, розроблена, щоб максимально ускладнити компіляцію. Компілятор таки існує.

```
> v
v ,,,,,"Hello"<
>48*, v
v ,,,,,"World!"<
>25*,@
```

<sup>47</sup>Наприклад, CINT – <http://www.hanno.jp/gotom/Cint.html>, частково підтримує C++.

<sup>48</sup>Наприклад, cling – <https://root.cern.ch/cling>. Ще один схожий проект – Ch, <http://www.softintegration.com>.

## Інтерпретація vs компіляція – 2

- Певна відмінність між компільованими та інтерпретованими мовами, все ж, існує.
- Творці інтерпретованих мов, зазвичай, зосереджуються на простоті написання коду нею, можливості його легко запускати (без перекомпіляції) тощо – беручи на себе більше турботи про технічні аспекти (наприклад, типи змінних), ціною втрати продуктивності.
- Розробники компільованих мов – на продуктивності програм, написаних відповідною мовою.
- Класичний приклад тут Python – писати на ньому програми просто та зручно, однак, ефективність, в порівнянні із аналогічним кодом на C++, нижча на порядок-півтора<sup>49</sup>.
- Щоправда, байткоду Java це не дуже стосується. JVM є інтерпретатором, але вона інтерпретує машинну мову, розроблену спеціально для того, щоб бути швидкою та універсальною, а не зрозумілою та простою для написання людиною<sup>50</sup>.

<sup>49</sup>Як наочний приклад, див. "[Python One Billion Row Challenge](#)". Також: "[Why Python Is So Slow \(And What Is Being Done About It\)](#)" та "[A 100x speedup with unsafe Python](#)".

<sup>50</sup>Тому втрати швидкодії мінімальні – на сучасних JVM код повільніший в 1.1-2 рази, а за певних умов (хоча, про них більше говорять, ніж зустрічають на практиці), може бути швидшим за код на C/C++, завдяки динамічній перекомпіляції із оптимізацією під конкретні потоки даних, з якими працює код. Щоправда, трапляються ситуації, коли повільніший в 5 і більше раз.

# Посилання

- Есей: "[Beating the compiler](#)" – компілятор зазвичай ефективніший, ніж вручну написаний код. Звичайно, SIMD-код людина часто все ще краще пише. Але тут автор показує, як обігнати компілятор, пишучи інтерпретатор на асемблері.

# Мікроархітектура

- На центральному процесорі історія із трансляцією не закінчується.
- Часто команди, що поступають в процесор та належать до його системи команд – ISA<sup>51</sup>, не виконуються безпосередньо, а інтерпретуються всередині процесора, так званим *мікрокодом*, із використанням своєї системи *мікрокоманд*, *мікроасемблером* тощо.

---

<sup>51</sup>Instruction set architecture, не плутати із шиною ISA (Industry Standard Architecture), популярною в 80-х та на початку 90-х!

# Емуляція комп'ютерної системи

- На мікроархітектурі історія теж не закінчується.
- Код – операційна система, разом із прикладними програмами, може виконуватися в *емуляторі*, який, у свою чергу, виконується на зовсім іншій платформі.
- Емулятор комп'ютерної системи – програма (чи, рідше, пристрій), яка дозволяє одній машині поводитися як інша, часто – не пов'язана.
- При чому, у віртуальній машині може бути запущена нова віртуальна машина, і так можна продовжувати, поки не вичерпається пам'ять або можливості емуляторів.
- Звичайно, практично такі "матрьошки" використовуються не часто, але, враховуючи поточну популярність хмарних обчислень – трапляються. А ще недавно дана цитата була жартом (<https://bash.im/quote/59325>):  
*"Флудити то флудити! :) StarControl2, запущений в DosBox, під X в Debian, який запущений в VMWare, яка – в WinXP. До кого звертатися з приводу звуку, що не працює?"*  
*"До санітарів"*

## Емуляція пристроїв

- Також, окрім цілої комп'ютерної системи може емулюватися окрема її частина.
- ◇ Скажімо, XT-IDE емулює жорсткий диск для ранніх IBM-сумісних комп'ютерів, позбавляючи потреби для експериментів із ними використовувати все більш рідкісні прилади із складною механікою, яка схильна виходити з ладу.
- ◇ Інший приклад – pdf-принтер, який дозволяє "надрукувати" будь-який документ у pdf.
- ◇ Ще один: віртуальні диски, змонтовані із образів – .iso, .ima чи інших.
- ◇ Більш маргінальний приклад: CPU Apple M1 емулює систему команд x86. При чому, емулює також більш строгу модель узгодження пам'яті<sup>52</sup> x86. Іншими словами, він може перейти в режим, в якому буде поводитися як x86.
- Ремарка: симуляція – це зовсім інше поняття, комп'ютерна симуляція – моделювання якогось процесу – у фізиці, біології, соціології тощо, за допомогою комп'ютера.

<sup>52</sup>Детальніше, що таке моделі узгодженості пам'яті, буде розглядатися в курсі АКС. 

# Віртуалізація

- Суміжним до емуляції поняттям є **віртуалізація**.
- Однією із найважливіших функцій операційної системи, критичною для успіху самої ідеї операційної системи, є *віртуалізація* ресурсів комп'ютера – процесорів, пам'яті, периферійних пристроїв – дисків (та інших пристроїв збереження), мережевих карт, клавіатур, мишок, дисплеїв, принтерів тощо.
- Сучасні "великі" операційні системи дозволяють кожній програмі вважати<sup>53</sup>, що та одноособово працює із процесором, пам'яттю, периферійними пристроями.
- Ціллю такої віртуалізації – ізоляція прикладних програм від апаратних пристроїв.
- А також, ізоляція реалізації операційних систем від подробиць функціонування апаратних пристроїв. Це досягається **драйверами пристроїв**.

---

<sup>53</sup>Звичайно, до певної межі. Зрозуміло, що адекватні програми повинні враховувати реальний стан справ»

- Важливість такої віртуалізації можна показати на прикладі MS-DOS, найпопулярнішої<sup>54</sup> ОС 1980-х.
- З сучасної точки зору, це була дуже примітивна операційна система. Однак, її слід відзначити, як першу популярну ОС<sup>55</sup> із динамічним завантаженням драйверів периферійних пристроїв – до того було прийнято вкомпільовувати їх у ядро ОС.
- То знаковою є цитата із "MS-DOS: Technical Reference Encyclopedia", 1986 року<sup>56</sup>: *"To be successful over the long term, Microsoft's new operation system would have to be flexible enough to accommodate change. Toward that end, Microsoft sought to make its operations system less dependent on any specific hardware configuration—**that is, to virtualize its operating system in every way it could.** The first steps were device-independent input and output, variable record lengths, relocable program files, and a replaceable command processor."* .

---

<sup>54</sup>В якомусь сенсі.

<sup>55</sup>Автор буде вдячний за інші ранні приклади таких ОС.

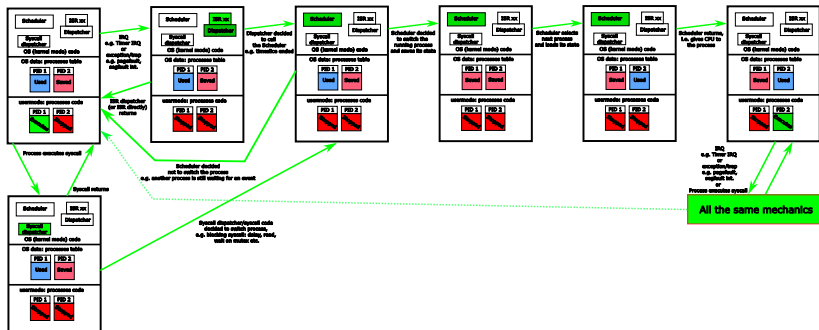
<sup>56</sup>Виділення зроблене для цієї презентації.

- Для віртуалізації процесора **для прикладних процесів** ОС використовує поняття процесу та багатозадачності.
- ◊ Багатозадачність буває двох видів: кооперативна (cooperative) та витісняюча (preemptive).
- ◊ Витісняюча багатозадачність характерна для сучасних ОС: процеси призупиняються примусово і диспетчер (scheduler) ОС вибирає наступний процес для виконання. Для призупинки процесу використовується переривання від таймера – за його допомогою диспетчер отримує керування та можливість здійснити переключення.
- ◊ Кооперативна багатозадачність – процес має сам віддати процесор, сам явно звернутися до диспетчера, для переключення на інший процес. Ранні багатозадачні операційні системи, типу 16-бітовою MS Windows<sup>57</sup> користувалися нею для оптимізації, зараз ця ідея повернулася, знову ж – для оптимізації, у вигляді ідеї співпрограм (coroutines).
- У випадку витісняючої багатозадачності кожен процес може вважати, що він єдиний користується процесором, а ОС здійснює менеджмент доступу до CPU різними процесами – **віртуалізує процесор**, виділяючи частини часу роботи CPU кожному із них.
- ◊ Для кооперативної багатозадачності процеси повинні регулярно віддавати процесор, але у всьому решта теж можуть вважати, що процесор повністю їх.
- Детальніше подробиці розглядаються на курсі ОС.

---

<sup>57</sup>Які швидше були операційним середовищем, але MS-DOS+Windows можна вважати повноцінною ОС. ▶

# Скінчений автомат витісняючої багатозадачності



- Випадок одного CPU, на якому виконується два процеси.
- ◇ Кожен прямокутник умовно зображає стан комп'ютера. Верхня його частина – код ядра операційної системи, середня – стан її структур даних, важливих для нашого розгляду, нижній – стан користувацьких процесів.
- ◇ Зеленим показано, який код на ньому зараз виконується.
- ◇ PID – Process ID, унікальний ідентифікатор процесу.
- Спрощуючи, dispatcher відповідає за низькорівневі операції по переключенню процесів, Scheduler – високорівневий код, який визначає коли і на який процес переключатися.
- Як потрібно змінити цю схему для кооперативної багатозадачності?

# Віртуальна пам'ять

- Віртуалізація пам'яті здійснюється з використанням механізмів віртуальної пам'яті<sup>58</sup>.
- Апаратний пристрій, який здійснює трансляцію та розмежування прав доступу, називається MMU<sup>59</sup> – Memory Management Unit. Зараз це, зазвичай, частина CPU.
- Хоча детальніше ми віртуальну пам'ять будемо розглядати на курсах АКС та ОС, це механізм, навколо якого побудовано все функціонування сучасних ОС. Тому коротко познайомимось із ним.
- Основна ідея – **трансляція**.
  - ◊ В сучасних системах пам'ять розбита на сторінки фіксованого розміру. Зазвичай це 4 Кб<sup>60</sup>.
  - ◊ Існує спеціальна **таблиця сторінок** (page table), яка керується операційною системою. Вона встановлює відповідність логічних адрес фізичним адресам, окремо для кожної сторінки.
  - ◊ Коли процес звертається за якоюсь адресою, ця **логічна адреса** за допомогою таблиці сторінок **трансляється у фізичну адресу**, яка і буде виставлятися на шині пам'яті.
  - ◊ У таблиці сторінок може бути вказано – сторінка відсутня у фізичній пам'яті.

<sup>58</sup>Зі всіх видів віртуалізації, що здійснює ОС, певне, хіба тут слово віртуалізація вживається явно.

<sup>59</sup>Його варіант, популярний в мікроконтролерах, який забезпечує лише захист, без можливості трансляції сторінок, називається MPU – Memory Protection Unit.

<sup>60</sup>Але є багато-багато нюансів!

# Приклад трансляції

mov EAX, [0x0002204]

Virtual (linear) address

|                     |    |                |   |
|---------------------|----|----------------|---|
| 31                  | 12 | 11             | 0 |
| Virtual page number |    | Offset in page |   |
| 0x00002             |    | 0x204          |   |

Page index

0x00000

0x00001

0x00002

0xFFFFF

| PAGE TABLE |             |                                 |
|------------|-------------|---------------------------------|
| Present    | Other flags | Physical page number<br>20 bits |
| 0          |             |                                 |
| 1          |             | 0x01005                         |
| 1          |             | 0x00012                         |
| .          | .           | .                               |
| 0          |             |                                 |

Page table base register  
(Physical address)

0x0012204  
Physical address  
on memory bus

|                      |    |                |   |
|----------------------|----|----------------|---|
| 31                   | 12 | 11             | 0 |
| Physical page number |    | Offset in page |   |
| 0x00012              |    | 0x204          |   |

## Використання swar на додачу до фізичної пам'яті

- Якщо відбувається звернення до змінних, що знаходяться на сторінці, яка помічена як відсутня, виникає так зване **сторінкове переривання** (page fault).
- В ISR (обробнику) цього переривання ОС може перевірити, чи це звертання коректне, і якщо так – завантажити сторінку із сховища (диску) у RAM та надати можливість коду, який викликав page fault, повторити спробу – вказати CPU повторно виконати команду, що спричинила page fault.
- Для користувацьких процесів ця процедура повністю прозора – за винятком доволі помітної часової затримки.
- Ситуація відсутності сторінки може виникати у двох випадках:
  - ◇ Сторінку ще не було завантажено, але ОС знає, де її взяти, або вона була прочитана із файлу та залишалася незмінною.
  - ◇ Або (змінену) сторінку було витіснено із фізичної пам'яті на диск – у так звані **файли підкачки** (swar file) чи **розділи підкачки** (swar partition).
- Ще одна функція MMU – перевірка прав доступу до сторінок. Зазвичай, права є наступні: user/kernel чи supervisor, execute/noExecute, read/write/no access.

# Використання віртуальної пам'яті – 1

- Механізми віртуальної пам'яті дають можливість ОС значно більш гнучко керувати фізичною пам'яттю.
- ◇ Наприклад, завантажувати лише ті сторінки коду чи даних, що реально використовуються.
- ◇ Або викидати із фізичної пам'яті ті сторінки, що зараз не потрібні, або менш потрібні за інші.
- ОС встановлює для кожного процесу свою таблицю сторінок.
- ◇ Завдяки цьому кожен процес має свій, незалежний від інших, адресний простір.
- Ще одне використання – одну і ту ж сторінку фізичної пам'яті відображати у пам'ять кількох процесів. Це дозволяє економити фізичну пам'ять – дуже цінний ресурс.
- ◇ Наприклад, якщо динамічна бібліотека спільного використання використовується кількома процесами, її код можна завантажити в фізичну пам'ять лише раз – не тримати багато копій<sup>61</sup>.
- ◇ Ще один приклад – якщо є кілька копій одного процесу, всі вони можуть використовувати одні і ті ж сторінки коду.

<sup>61</sup>Це важлива перевага динамічних бібліотек над статичними.

## Використання віртуальної пам'яті – 2

- ОС також може відображати ті ж фізичні сторінки із даними у кілька процесів – це дозволить їм швидко обмінюватися даними, однак потребуватиме синхронізації<sup>62</sup>.
- Сучасні ОС також можуть програмно відображати вміст файлів на пам'ять – звертання до файлів емулюються роботою із пам'яттю. Якщо вас ця ідея заплутує – зачекайте до курсу ОС<sup>63</sup>.

---

<sup>62</sup>Детальніше це буде вивчатися на АКС і ОС. Ключове слово – міжпроцесний семафор.

<sup>63</sup>Це теж називають вводом-виводом, відображенням на пам'ять, але не слід плутати із аналогічною апаратною концепцією, коли пристрої під'єднані до шини пам'яті.

## Як не варто говорити про віртуальну пам'ять

- Формально правильним, але надміру вузьким і дуже стереотипним є твердження: "віртуальна пам'ять дозволяє завантажувати програм більше, ніж може поміститися в фізичній пам'яті". Проаналізуємо його:
- ◇ Пам'ять, що використовується процесом, можна грубо розділити на дві частини: так званий *working set*, пам'ять, що активно використовується на цьому етапі виконання процесу; та вся решта пам'ять – та, що була потрібна під час ініціалізації і більше ніколи не буде використовуватися, та, що належить рідко використовуваним функціям, та, яка була потрібна цілу вічність<sup>64</sup> тому і знадобиться ще через вічність і т.д.
- ◇ Віртуальна пам'ять дозволяє ефективно використовувати наявну фізичну пам'ять, витісняючи із неї рідко потрібні код та дані, та підвантажуючи їх за потреби.
- ◇◇ Змінені дані при цьому будуть зберігатися у так звані файли підкачки (*swap file*) чи розділи підкачки (*swap partition*), а незмінні – скажімо, код програм, викидається і буде читатися із оригінальних файлів.

---

<sup>64</sup>Скажімо, секунд 10 тому.

- ◇ Враховуючи, що на сучасному користувацькому комп'ютері можуть бути запущені сотні фонових процесів, у такому сенсі – формально належної процесам, але, фактично – не використовуваної пам'яті, справді може бути більше, ніж є фізичної.
- ◇ Але більш коректно казати що тут віртуальна пам'ять забезпечує виконання **частково завантажених** програм.
- ◇ Якщо ж сумарний working set всіх активних процесів є більшим за фізичну пам'ять – то справді, формально, віртуальна пам'ять дозволить працювати цим процесам одночасно.
- ◇◇ Однак, оскільки на сучасних комп'ютерах розрив між швидкодією RAM та сховища (дискової пам'яті) складає 2-3 порядки, іноді – більше, продуктивність комп'ютерної системи при цьому паде в тисячі раз – і якогось прогресу у виконанні роботи не відбувається.
- ◇◇ Відбувається так званий **трешинг** – блоки пам'яті кожного із процесів постійно циркулюють між диском і RAM: переключилися на процес 1, потрібні йому сторінки в swp, підкачали, при цьому, оскільки пам'яті мало – витіснили в swp сторінки процесу 2, ледве почали працювати – переключилися на процес 2, його підкачали, процесу 1 – витіснили і так по колу.

- ◇ Хоча короткочасно ситуації трешингу, описані вище, не створюють великих проблем, під час нього комп'ютерна система стає дуже неефективною.
- ◇◇ В минулому – коли ця стереотипна фраза викристалізувалася, розрив між швидкодією RAM і storage був суттєво меншим.
- Ще одна важлива роль віртуальної пам'яті – релокації, можливість вільно перетасовувати відображення логічних адрес на фізичні, CoW-оптимізації<sup>65</sup>, оптимізації збірників сміття тощо.
- Таким чином, "віртуальна пам'ять дозволяє завантажувати програм більше, ніж може поміститися в фізичній пам'яті", впускає все різноманіття її використання – див. попередні слайди та акцентує увагу на найменш важливому її аспекті.
- Побутовою аналогією до цієї фрази є щось таке: "Квартира – фортеця, яка захищає нас від нападів ворогів". Це твердження правдиве. Але дуже не повне, а акценти розставлені дуже специфічно.

---

<sup>65</sup>Copy on Write.

# Віртуалізація вводу-виводу

- Пристрої вводу-виводу дуже різноманітні, тому віртуалізуються по різному.
- Мережева карта чи карти віртуалізуються за допомогою поняття сокету – кінцевої точки обміну даними між двома процесами.
- ◇ Віртуальні мережі – із віртуальною топологією, реалізовані поверх фізичних мереж, є дуже важливим аспектом побудови центрів обробки даних, однак сильно виходять за межі нашої тематики, тому їх тут не будемо розглядати.
- Клавіатура сучасними GUI віртуалізується за допомогою поняття фокусу – ввід потрапляє у те вікно, і той віджет вікна, що зараз в фокусі.
- Дисплей<sup>66</sup> віртуалізується за допомогою поняття "вікно" – прямокутної ділянки екрану<sup>67</sup>, видимістю якого керує підсистема GUI, скажімо – віконний менеджер.

<sup>66</sup>Віртуалізація мишки – дещо складніша тема, пов'язана із реалізацією обробки повідомлень від неї, тому тут ми її не обговорюємо.

<sup>67</sup>Зазвичай, не-прямокутна форма вікна досягається прозорістю його окремих частин.

## Віртуалізація сховища (storage) чи дискової пам'яті

- Віртуалізація storage – дискової пам'яті, дворівнева:
- ◇ Фізичний диск<sup>68</sup> може містити кілька логічних **розділів (partitions)**<sup>69</sup>, кожен з яких може вважатися ОС окремим **логічним диском**. Їх також іноді називають volume, том.
- ◇ Сучасні ОС прикладним програмам надають доступ до диску за допомогою абстракції **файл**.
- ◇ Також кілька фізичних дисків можуть бути об'єднані у один логічний, за допомогою технологій, таких як RAID<sup>70</sup> чи LVM<sup>71</sup>. Див. також файлову систему zfs.
- ◇ У великих датацентрах використовуються інші підходи до віртуалізації сховища, які дозволяють мати фізичні диски, що використовуються кількома комп'ютерами одночасно.
- З іншого боку, файли можуть бути представлені як диски. В Linux це виконується з допомогою loopback device, під Windows потребує сторонніх програм, типу OSFMount, Alcohol 120% чи Daemon Tools.

<sup>68</sup>Хоча SSD формально не містять реальних обертових дисків, на відміну від HDD, їх теж прийнято так називати.

<sup>69</sup>Сучасний спосіб їх організації — GPT, GUID Partition Table; попередній, застарілий, але все ще часто використовуваний — MBR, Master Boot Record.

<sup>70</sup>Redundant Array of Independent Disks.

<sup>71</sup>Logical volume manager.

# Віртуалізація комп'ютерних систем

- Віртуальні машини – дуже популярний інструмент.
- Ми розглянули один спосіб їх реалізації – емуляцію.
- Виконання програми для PDP-11 на системі із процесором сімейства x86 вимагатиме повної емуляції.
- Однак, повна емуляція програми для x86 на ньому ж, яка є доволі затратною, ще й не дуже потрібна.
- Для подальшого розгляду, назовемо операційну систему, під якою запущено віртуальну машину, **host operating system**.
- Запущені ж у віртуальній машині (будь-якої природи) ОС називатимемо **guest operating systems**.

- Host OS безпосередньо керує всіма ресурсами комп'ютера<sup>72</sup>.
- З гостьовою операційною системою складніше:
- ◊ Більшість команд головний процесор може виконувати безпосередньо.
- ◊ Команди, які впливають на глобальний стан системи – здійснюють ввід-вивід чи керують таблицями сторінок тощо, так виконувати не можна – це завадить функціонуванню хостової ОС.
- Однак, завдяки механізму розмежування привілеїв, підтримуваних CPU, розділити ці види команд просто: вони є привілейованими.
- Таким чином, код, що реалізовує віртуальну машину, може запустити її код з привілеями user mode команд.
- ◊ Звичайні команди будуть виконуватися CPU безпосередньо.
- ◊ Привілейовані ж викликатимуть exception порушення захисту.
- ◊ Код віртуальної машини, за допомогою спеціального драйвера, перехоплює їх – ці виключення, та для гостьової системи робиться вигляд, що привілейована команда виконала свою роботу – емулюються відповідні операції.

---

<sup>72</sup>Під час роботи гіпервізора це не зовсім правда, але зійде як прийнятне наближення.

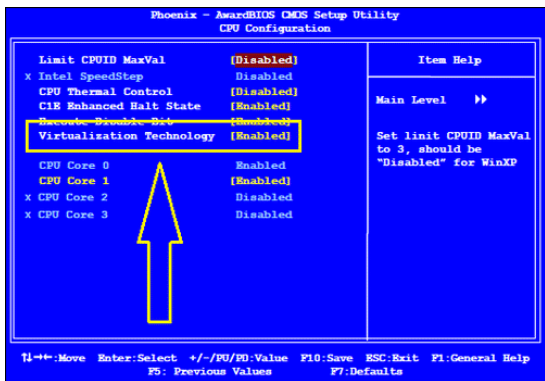
- Детальні вимоги придатності комп'ютерної системи до віртуалізації викладені у статті <http://www.cs.cornell.edu/courses/cs6411/2018sp/papers/popek-goldberg.pdf>:
- ◇ Виконання коду під керуванням менеджера віртуальних машин повинно бути еквівалентним виконанню на апаратному забезпеченні безпосередньо.
- ◇ Менеджер віртуальних машин повинен ексклюзивно керувати ресурсами, які віртуалізуються – щоб могли забезпечити ізоляцію віртуальних машин.
- ◇ Абсолютна більшість команд має виконуватися безпосередньо.
- ◇ Всі команди, які впливають на стан машини, повинні викликати виключення у привілейованому режимі.
- ◇ Не повинно бути команд, які "мовчки" по-різному поведуться в привілейованому і в користувачькому режимах.
- Цитата з статті "[There's No Such Thing as a General-purpose Processor](#)"<sup>73</sup>: *The important requirements for running an operating system <...> are similar to those enumerated by Gerald Popek and Robert Goldberg for virtualization. This is not surprising, as an operating-system process is effectively a virtual machine, albeit with a very high-level set of virtual devices (file systems rather than disks, sockets rather than Ethernet controllers).*

---

<sup>73</sup>Стаття цікава! Але не те щоб я був згоден з її назвою.

# Віртуалізація x86 – 1

- Як завжди, є нюанси: популярна зараз архітектура x86 – жахлива з точки зору віртуалізації.
- У ній присутні команди, які "мовчки" по-різному виконуються в kernel mode і user mode.
- Так склалося історично. Сучасні версії цих процесорів підтримують розширення, що виправляють цю проблему, разом із проблемами прозорого розмежування ресурсів.
- ◊ На жаль, різні у різних виробників: AMD-V та Intel VT.
- Іноді їх підтримку доводиться вмикати в BIOS Setup чи аналогічних UEFI інструментах:



## Віртуалізація x86 – 2

- До появи розширень для підтримки віртуалізації на x86, рішення типу VMware для віртуалізації x86 використовували цікаві трюки із динамічною бінарною трансляцією – частина гостьових ОС переписувалася на льоту, замінюючи проблемні інструкції<sup>74</sup>.
- Ще один приклад спеціалізованого "гіпервізора" – зловживання ACPI, для швидкого переходу між Windows і (обмеженим) Linux на старенькому ноутбукі, ≈ 2009: ["Hell Never Ends On x86: The Hyperspace Story, part 1"](#)<sup>75</sup> та ["part 2"](#). Назва в технології красива: Phoenix Hyperspace. Продовження: ["Hell Never Ends On x86: The Hyperspace Story, Continued, Sort Of"](#) – тут вже не так цікаво, умовно, календар-органайзер на UEFI.

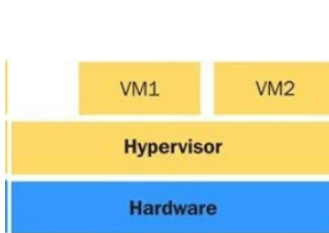
---

<sup>74</sup>"Хто сказав байткод?"

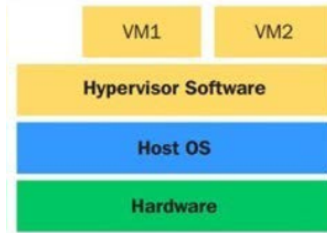
<sup>75</sup>Назва умовна – у автора ця стаття не підписана

# Virtual Machine Manager (VMM) – hypervisor

- Програмне забезпечення, що реалізовує віртуальні машини, називається **Virtual Machine Manager (VMM)**, або **гіпервізор**.
- Гіпервізори поділяють на два види<sup>76</sup>:
  - ◇ **Гіпервізор першого роду** (type 1) виконуються безпосередньо на апаратному забезпеченні, без використання хостової ОС. Їх також називають bare-metal, embedded або native hypervisors.
  - ◇ **Гіпервізор другого роду** (type 2) виконується поверх хостової ОС, покладаючись на її драйвери для взаємодії із апаратурою.



Type 1



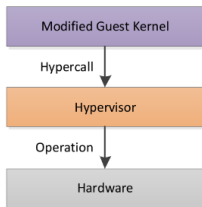
Type 2

<sup>76</sup> Назви дуже вже ненаочні, але – так встановилося.

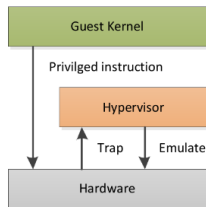
# Паравіртуалізація

- Вище розглядалася віртуалізація ОС, які нічого не знають про це – не модифікованих для підтримки віртуалізації.
- Плюси цього зрозумілі – можна користуватися саме тими варіантами ОС, що потрібні.
- Однак, є і мінуси – це повільніше, падає загальна продуктивність, зростають часи відгуку.
- Тому популярним є і альтернативний підхід: ОС, замість безпосередньо виконувати привілейовані команди, викликає певні функції<sup>77</sup>, реалізовані гіпервізором.
- Такий підхід називають **паравіртуалізацією**.

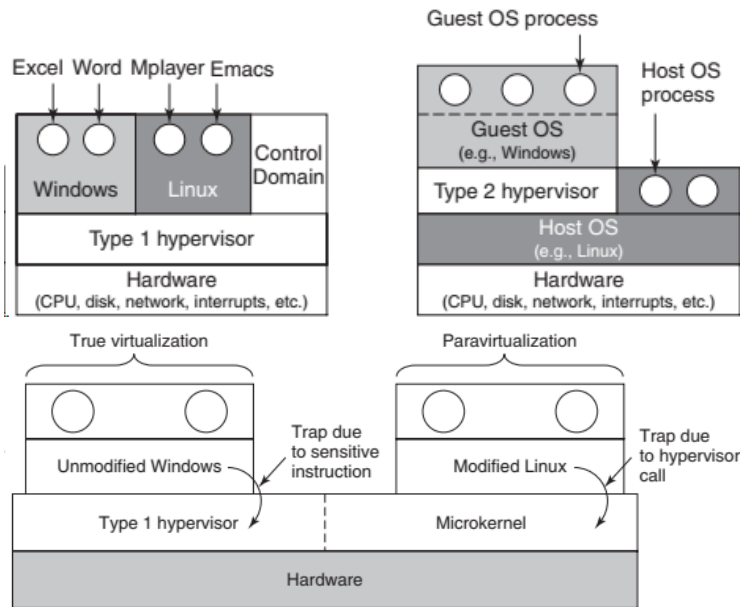
Para-virtualization



"Classical" Full-virtualization

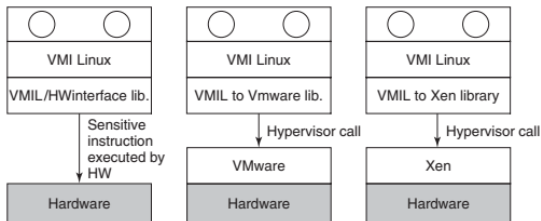


<sup>77</sup>Іноді такі виклики, по аналогії із syscalls – звертанням до ОС, називають hypercalls.



- Альтернативні ілюстрації із книги Таненбаума.

- Ідею паравіртуалізації можна розвинути до гіпервізор-агностичного ядра ОС, прилінковуючи відповідну бібліотеку викликів гіпервізора, чи звертань до апаратури безпосередньо, вже під час запуску ОС:



- Ядро Linux підтримує такий підхід.

- Ефективна віртуалізація сучасних комп'ютерних систем – дуже складне завдання із багатьма компонентами. Детальний їх розгляд сильно виходить за межі нашого курсу. Тому, для допитливих студентів, лише наведемо пов'язані терміни:
- Shadow page tables; Nested Page Tables (AMD)/Extended Page Table(Intel).
- I/O MMU<sup>78</sup>; device pass through; interrupt remapping; VT-d (Intel); AMD-Vi (AMD); System Memory Management Unit (ARM); "Single root I/O virtualization (SR-IOV)" (Intel).
- VirtIO, Vhost, VFIO, Vhost-user, VFIO-mdev, vDPA, VDUSE – набір технологій та стандартів для спрощення емуляції периферії. Їх історія: "The Evolution of Network Virtualization Technologies in Linux"
- "vDPA kernel framework part 3: usage for VMs and containers" – детальний есей з ілюстраціями.

---

<sup>78</sup>Часто пишуть IOMMU – адже мати можливість переадресовувати відображення пристроїв вводу-виводу на пам'ять теж корисно.

## Інші види віртуалізації

- **OS-level virtualization** – в межах однієї операційної системи виділяються ізольовані простори імен чи пісочниці, які не знають одна про одну.
  - ◇ Зокрема, вони бачать лише процеси, що належать тому ж простору імен, їх кореневою файловою системою є, часто, директорія, вказана при створенні, доступним їм може бути лише підмножина пристроїв.
  - ◇ Приклад: контейнери Linux<sup>79</sup>, типу докера; FreeBSD jail тощо.
- **Віртуалізація системних викликів чи їх аналогів.** Wine дозволяє виконувати програми Windows під Linux. Він здійснює трансляцію викликів Win API в системні виклики POSIX та надає іншу інфраструктуру, на яку покладаються програми Windows – скажімо, реєстр.
  - ◇ В зворотному напрямку: cygwin, [midipix](#), WSL1 – див. далі.
- **Application virtualization** (віртуалізація програмного забезпечення) – енкапсуляція у своєрідному контейнері окремого процесу.
  - ◇ Суміжною технологією є **Workspace virtualization** – енкапсуляція не одного процесу, а цілого комплексу процесів, що реалізують робочий стіл GUI.

<sup>79</sup>Під Linux будуються навколо chroot, просторів імен ядра та cgroups.

- **Desktop virtualization** – віртуалізація робочого середовища GUI від клієнтів, що із ним працюють, зокрема – мережева.
- ◇ Популярна в корпоративному секторі – дозволяє працювати із застосунками, що виконуються на корпоративних серверах за допомогою ноутбуків, планшетів, смартфонів тощо, із будь-якого місця і без додаткових проблем у випадку втрати цих пристроїв.
- ◇ Суміжною технологією є User virtualization – профілі юзерів зберігаються та керуються окремо від конкретних ОС та комп'ютерів, на яких із ними працюють.



## WSL1

- Приклад: Windows Subsystem for Linux 1 (WSL 1). Зворотне до Wine – транслює системні виклики POSIX у виклики Windows.
- ◇ Ідея базується на підтримці Windows так званих підсистем.
- ◇ "Рідним" API сучасних Windows є Native API, не особливо документоване. Поверх нього реалізовано WinAPI – як одну із підсистем.
- ◇ Раніше існували підсистеми POSIX та OS/2, які до кінця 1990-х були викинуті.
- ◇ Для реалізації WSL1 використано так звані **пікопроцеси** – в адресний простір не відображається нічого звичного для процесів Windows. Те ж стосується потоків, створених в таких процесах.
- ◇ Спеціальний pico provider отримує всі системні виклики та виключення від коду процесу. Далі він перенаправляє їх решті ядра.
- ◇ Іншою важливою використаною ідеєю є "library OS" – ОС, модифікована так, щоб працювати як бібліотека всередині такого пікопроцесу.
- ◇ Не підтримує драйверів (ядра Linux як такого немає), безпосередню роботу із GPU (CUDA)<sup>80</sup>, 32-бітовий код. Зате швидкість дискового вводу-виводу – нативна.
- ◇ Існує також експериментальна реалізація Windows NT як library OS – user-mode NT kernel, NTUM.
- Детальніше див.: [Drawbridge](#) – дослідницький проект, з якого ця ідея виросла. Їх репозиторій: [pico-toolbox](#).
- ◇ [Windows Subsystem for Linux](#) – огляд WSL. Зокрема, там детально описано, як здійснюється віртуалізація вводу-виводу та доступу до дисків.
- ◇ [The Linux kernel hidden inside windows 10](#).
- ◇ ["Pico Process Overview"](#) та ["WSL File System Support"](#) від MS.
- [Personality](#) – системний виклик Linux із схожими функціями.

<sup>80</sup>В теорії це можливо.

# WSL2

- Windows Subsystem for Linux 2 (WSL 2) – це віртуальна машина на базі гіпервізора Microsoft Hyper-V, тісно інтегрована у Windows.
- ◇ У ній виконується звичайний Linux, зі своїм ядром і т.д.
- ◇ Завдяки цьому можливим є використання своїх драйверів, звертатися до GPU (використовувати CUDA) тощо, але швидкість дискових операцій може страждати, оскільки вони реалізовані поверх мережевого стеку, з використанням 9P – Plan 9 Filesystem Protocol.
- ◇ "Windows Subsystem for Linux: The lost potential" – есей на тему, чому WSL1 – цікава ідея, і яка "загроза" їй від WSL2.
- В 2025 році WSL було викладено як Open Source: "The Windows Subsystem for Linux is now open source", [репозиторій](#).

- Ремарка щодо термінології: терміни, пов'язані із віртуалізацією часто мають різні вживання у різних академічних школах, різних компаніях тощо.
- Вище використовувалася термінологія у стилі книги Таненбаума "Сучасні операційні системи" та орієнтуючись на світ Linux, зокрема, термінологію, прийняту в KVM та QEMU, хоч із варіаціями.
- Через різномаїття вживання термінів, важливо розуміти відповідні ідеї та вміти, під час розмови, розпізнати термінологічну систему, використовувану співбесідником.
- Окрім згаданої книги Таненбаума<sup>81</sup>, див. "Mastering KVM Virtualization: Design expert data center virtualization solutions with the power of Linux KVM", 2nd Edition, by Vedran Dakic, Humble Devassy Chirammal, Prasad Mukhedkar, Anil Vettathu.

---

<sup>81</sup> Детальний її огляд див. <http://indrekis2.blogspot.com/2016/06/blog-post.html>.

## Підсумок – різна "глибина" віртуалізації

- Повна емуляція комп'ютерної системи
- ◇ В емульовану машину можуть "прокидуватися" (passthrough) деякі реальні пристрої.
- Віртуалізація ОС з використанням розмежування привілеїв<sup>82</sup>.
- ◇ Драйвери пристроїв і їх віртуалізацію забезпечує гіпервізор.
- ◇ Драйвери пристроїв надає host OS, віртуалізацію забезпечує гіпервізор.
- ◇ Один із двох попередніх варіантів, але окремі пристрої "прокинуті" ОС, що віртуалізується.
- Паравіртуалізація.
- Віртуалізація на рівні системних викликів, чи їх аналогів: Wine, WSL1, багато історичних прикладів.
- OS-level virtualization – контейнери, FreeBSD jail тощо.
- ◇ Application virtualization – те ж, для окремого процесу, Workspace virtualization – або для групи процесів, що створюють робочий стіл.
- Віртуалізація робочих столів користувачів та їх аккаунтів.

<sup>82</sup>Та інших засобів підтримки віртуалізації, типу I/O MMU.

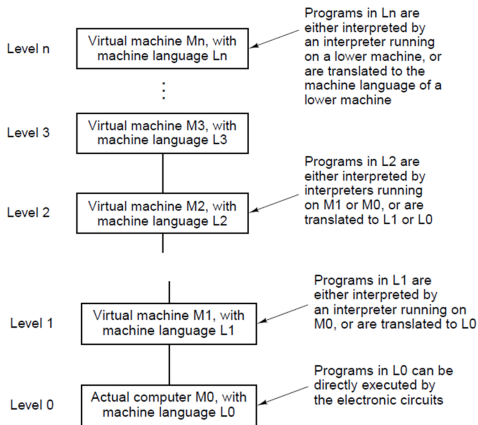
# Історія

- Першою системою із підтримкою повної віртуалізації, ймовірно є IBM CP/CMS 1967 року, нащадком якої є IBM VM (чи VM/CMS).
- ◇ Термін *hypervisor* запропонувала IBM в 1965, для мейнфрейма<sup>83</sup> IBM 360/65.
- ◇ Детальніше див.  
[https://en.wikipedia.org/wiki/VM\\_\(operating\\_system\)](https://en.wikipedia.org/wiki/VM_(operating_system)) та "An introduction to operating systems", Revised 1st ed., 1984, by Harvey M. Deitel.

---

<sup>83</sup>Есей на тему походження терміну: "[The origin and unexpected evolution of the word 'mainframe'](#)". ▶ ◀ ≡ ≡ ≡ ↺ 🔍 ↻

- Таким чином, наслідуючи Таненбаума, виконання програми на комп'ютері можна зобразити так:



- В різні моменти важливим є різний рівень деталізації – ми з тим ще стикатимемося.

- Одна із найскладніших проблем, що стоять перед С та С++ – різнорамітність платформ, на яких вона використовується.
- Хоча зараз 8-бітовий байт – норма<sup>84</sup>, а цілі від'ємні числа представляються у доповнювальному коді<sup>85</sup>, так було не завжди.
- Називаючи байтом мінімальну адресовану комірку пам'яті<sup>86</sup>, згідно книги Таненбаума "Structured Computer Organization":

| Computer         | Bits/cell |
|------------------|-----------|
| Burroughs B1700  | 1         |
| IBM PC           | 8         |
| DEC PDP-8        | 12        |
| IBM 1130         | 16        |
| DEC PDP-15       | 18        |
| XDS 940          | 24        |
| Electrologica X8 | 27        |
| XDS Sigma 9      | 32        |
| Honeywell 6180   | 36        |
| CDC 3600         | 48        |
| CDC Cyber        | 60        |

<sup>84</sup>Тривалі пошуки систем із байтом відмінної довжини майже не дали результатів – хіба туманні згадки про якісь пропріетарні DSP.

<sup>85</sup>Знову ж, винятків не вдалося знайти, але є надія на екзотичні DSP.

<sup>86</sup>Як завжди, є багато нюансів і екзотичних винятків, але, в цілому, поняття добре означене.

- Те ж стосується і машинного слова – поняття ніби просте, базове, але довга історія розвитку заплутала і його.
- Ідейно, це – комірка пам'яті, найбільш зручна для CPU. Зазвичай, під цим мається на увазі, що її розмір рівний розміру регістрів CPU.
- Давні архітектури використовували різні хитрі трюки – як для оптимізації, так і через обмеження технологій чи не повного усвідомлення наслідків<sup>87</sup>. Це породило безліч винятків. Іноді і зараз трапляються експерименти із екзотичними підходами.
- Розвиток конкретної архітектури, із дотриманням зворотної сумісності, породив дуже хитрозакручених монстрів. Приклад – на наступному слайді.
- Див. також [https://en.wikipedia.org/wiki/Word\\_\(computer\\_architecture\)#Table\\_of\\_word\\_sizes](https://en.wikipedia.org/wiki/Word_(computer_architecture)#Table_of_word_sizes) – зразу можна згадати, що були ще й десяткові машини<sup>88</sup>.

---

<sup>87</sup>Остання причина є важливою значно рідше, ніж може здаватися – інженери кожної епохи змушені працювати, враховуючи її технологічні обмеження, не очевидні їх наступникам.

<sup>88</sup>Не знаю, чи на них використовувалося С, але якісь рудименти підготовки для цього в стандартах трапляються, див, наприклад, члени шаблону `std::numeric_limits`.

## Приклад – історія машинного слова x86

- З'явилася дана архітектура (ISA), як 16-бітова – процесори 8086/8088.
- При тому, на рівні асемблера, сумісна із 8-бітовими попередниками – 8008 та 8080. (Див. також додаток.)
- Мала 16-бітові регістри, тому, природно, (машинним) словом, word, назвали комірку, розміром 16 біт, 2 байти.
- Потім, з приходом 80386, архітектура стала 32-бітовою – зберігаючи сумісність із 16-бітовим кодом.
- Тому, хоча регістри в 32-бітовому режимі – 32-бітові, терміном "слово" називають 16-бітову комірку, а 32-бітову – подвійним словом, dword.
- З приходом 64-бітового розширення<sup>89</sup>, стало ще веселіше. Регістри 64-бітові, це "четвірне" слово, qword.
- Однак, хоча регістри 64-бітові, зроблено було це не так заради цілих чисел, як заради вказівників. Тому базове ціле число (int) залишилося 32-бітовими. А архітектура використовує дещо неочевидні для невтаємничених (але очевидні фахівцям) правила маніпуляції верхньою частиною регістрів.
- Більше прикладів на Wiki: [Word \(computer architecture\)](#).

<sup>89</sup>x86-64, не плутати із вже майже мертвою IA64.

- Есей про маніпуляції з "розміром" вказівників, та ілюстрація, чому на 64-бітових системах int залишено 32-бітовим: [Pointer Compression in V8](#).
- Детальніше див. [https://en.wikipedia.org/wiki/64-bit\\_computing#64-bit\\_data\\_models](https://en.wikipedia.org/wiki/64-bit_computing#64-bit_data_models) – LLP64/LP64.

| short | int | long | ptr | long long | Label                     | Examples                                                                                                                       |
|-------|-----|------|-----|-----------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| ...   | 16  | ...  | 16  | ...       | IP16                      | PDP-11 Unix (1973)                                                                                                             |
| 16    | 16  | 32   | 16  | ...       | IP16L32                   | PDP-11 Unix (1977); multiple instructions for long                                                                             |
| 16    | 16  | 32   | 32  | ...       | I16LP32                   | MC68000 (1982); Apple Macintosh 68K; Microsoft operating systems (plus extras for x86 segments)                                |
| 16    | 32  | 32   | 32  | ...       | ILP32                     | IBM 370; VAX Unix; many workstations                                                                                           |
| 16    | 32  | 32   | 32  | 64        | ILP32LL or ILP32LL64      | <b>Microsoft Win32</b> ; Amdahl; Convex; 1990 Unix systems; Like IP16L32, for same reason; multiple instructions for long long |
| 16    | 32  | 32   | 64  | 64        | LLP64 or IL32LLP64 or P64 | 64-bit systems<br><b>Microsoft Win64</b> (X64 / IA64)                                                                          |
| 16    | 32  | 64   | 64  | 64        | LP64 or I32LP64           |                                                                                                                                |
| 16    | 64  | 64   | 64  | 64        | ILP64                     |                                                                                                                                |
| 64    | 64  | 64   | 64  | 64        | SILP64                    |                                                                                                                                |
|       |     |      |     |           |                           | <b>Most Unix systems</b> (Linux, Solaris, DEC OSF/1 Alpha, SGI Irix, HP UX 11)                                                 |
|       |     |      |     |           |                           | HAL; logical analog of ILP32                                                                                                   |
|       |     |      |     |           |                           | UNICOS                                                                                                                         |

Ілюстрація з "<https://www.viva64.com/en/t/0012/>"

## Продовжуючи із архітектурними відмінностями

- Розмір байта вже стабілізувався на 8 біт. Цього вимагає і POSIX, і Windows. Доповнювальний код теж всезагальний. Винятки можуть бути поміж екзотичних DSP.
- Фактичний розмір машинного слова теж доволі обмежений. Зазвичай це 32 біти, рідше – 16 біт, для вказівників – 64 біти. (Винятки бувають, але рідко).
- Однак, пам'ять в системі може бути різних видів. Пам'ять коду, пам'ять даних, пам'ять на читання-запис та лише на читання і т.д. і т.п.
- Однак, CPU та підсистема пам'яті повинні мати право переставляти операції та звертання до пам'яті. Також, хотілося б надати це право компілятору.
- Однак, обчислювачів у системі може бути багато.
- Однак, логіка узгодження їхніх ієрархій пам'яті може бути найрізноманітнішою.
- І т.д. і т.п.

- Тому стандарти C і C++ часто дуже неконкретні у низькорівневих речах.
- Причин дві:
  - ◇ не змушувати компілятор робити щось неприродне для апаратного забезпечення – це буде коштувати втрат продуктивності,
  - ◇ залишити компілятору простір для оптимізації.
- Ціна – головний біль для програмістів.

# Модель машини C/C++ – 1

- Моделі машини C і C++ практично ідентичні<sup>90</sup>.
- Пам'ять – пронумерована послідовність байт. Кожен байт має унікальний номер, номери ці – впорядковані, можна говорити про попередні та наступні байти.
- Однак, нумерація може бути не неперервною – із проміжками. Конкретне представлення номерів байт (представлення вказівника) не фіксується, хоча, в цілому, допускається<sup>91</sup> конвертування у цілі типи достатнього розміру (`intptr_t`, `uintptr_t`) і назад.
- Пам'ять може бути різних видів – може існувати кілька "паралельних" адресних просторів<sup>92</sup>.
- Фундаментальні типи безпосередньо відображаються на комірки пам'яті відповідних розмірів – без всіляких посередників.
- Компілятор має велику свободу щодо переставляння різних операцій, доти, поки кінцевий результат не відрізнятиметься.

---

<sup>90</sup>Однак, на загал, це єдине місце де є сенс згадувати C і C++ разом – у всьому іншому, крім базового "синтаксичного ядра" це абсолютно різні мови.

<sup>91</sup>Але вважається поганою практикою! Виняток – хитрі оптимізації, зокрема – необхідні для реалізації lock-free структур даних. Причина – над вказівниками побітові операції безпосередньо виконувати не можна.

<sup>92</sup>На сучасних процесорах, це, наприклад, окремі адресні простори EEPROM та RAM AVR8, в минулому – сегменти x86. GCC безпосередньо підтримує їх: "[Named Address Spaces](#)".

- Зокрема, важливим є поняття `sequence point` (точка перебігу) – точки, де всі попередні операції із побічними ефектами вже відбулися, а всі наступні – ще не розпочалися. Компілятор не має права переставляти операції із побічними ефектами відносно точок перебігу.
- C++11 його деталізує<sup>93</sup>, щоб врахувати паралельні можливості сучасних машин: два вирази можуть бути впорядкованими – перший строго йде після другого (`sequenced before/sequenced after`); із невідомим впорядкуванням – спочатку один із них виконується, потім інший, але котрий перший – не визначено (`indeterminately sequenced`); виконання не впорядковане (`unsequenced`).
- `Sequence point` в С: оператор коми, логічні оператори `&&` та `||`, умова тернарного оператора, в кінці виразів, перед входом у функцію, після `return` із функції і ще ряд прикладів.
- Див. також "Abstraction and the C++ machine model" by B. Stroustrup (<http://www.stroustrup.com/abstraction-and-machine.pdf>), [https://en.wikipedia.org/wiki/Sequence\\_point](https://en.wikipedia.org/wiki/Sequence_point) та, скажімо, <https://blogs.msdn.microsoft.com/larryosterman/2007/05/16/the-c-abstract-machine/>.

---

<sup>93</sup>Разом із тим, оголосивши поняття `sequence point` застарілим.

# volatile

- Компілятор С працює з абстрактною машиною, яка співвідноситься з реальним (чи віртуальним) апаратним забезпеченням через "так, ніби" – результат має збігатися з обіцяним для абстрактної машини, але як реалізовано – на розсуд компілятора.
- ◊ Це відкриває йому широкий простір для оптимізації.
- Однак, там де ця абстракція "протікає", наприклад, при роботі з вводом-виводом, відображенням на пам'ять, на перетині з функціонуванням DMA, при роботі зі змінними, до яких звертається і IRQ чи обробник сигналів POSIX і інший код, потрібен спосіб повідомити компілятору про свої наміри.
- В С та С++ таким засобом є модифікатор `volatile`.
- Він змушує всі операції з такими змінними а) відбуватися, б) в тому порядку, в якому вони є в коді.
- ◊ Це єдине використання `volatile`. Більше ні для чого він не придатний, зокрема – шкідливий в багатопоточному коді<sup>94,95</sup>.
- ◊ При тому, операції з іншими змінними можуть переставлятися відносно операцій з `volatile`<sup>96</sup>
- Детальніше див. "Nine ways to break your systems code using volatile".

<sup>94</sup>В Java все зовсім по іншому – `volatile` це те, що ми в С/С++ називаємо `atomic`.

<sup>95</sup>Атомарністю робота з `volatile`-змінними не володіє. Та й бар'єром пам'яті для процесора не є, хіба для компілятора.

<sup>96</sup>Іншими словами, забігаючи наперед, `volatile`-змінні не мають семантики `acquire` чи `release`.

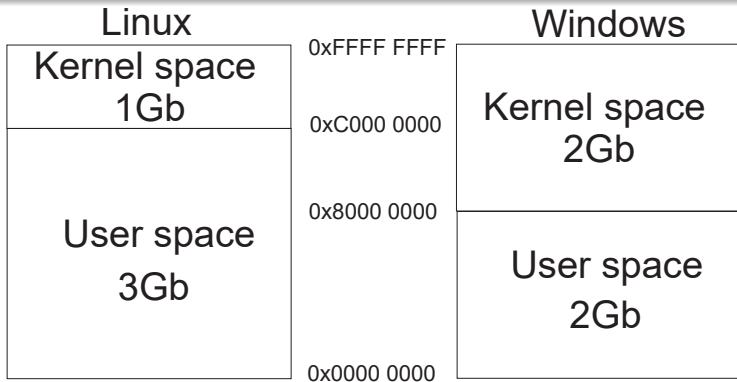
- Надалі ми орієнтуватимемося на модель пам'яті та архітектури x86 та ARM, іноді згадуючи AVR8.

## Карта пам'яті процесу

- 32-бітові Linux, Windows та деякі інші сучасні операційні системи надають кожному процесу 4Гб адресного простору.
- 64-бітові операційні системи теж виділяють весь доступний адресний простір.
- ◇ Формально це  $2^{64} = 16$  екзабайт, однак, оскільки фізичних адресних бітів менше – в сучасних реалізаціях їх 48..52, є нюанси.
- Адресні простори різних процесів<sup>97</sup> не перетинаються, знаходяться ніби в паралельних світах.
- ◇ Важливим винятком із цього є пам'ять спільного використання (shared memory), яка використовується для міжпроцесної взаємодії. Щоб скористатися цією можливістю, всі процеси-учасники мають явно зробити запит на відображення тієї ж фізичної пам'яті у їхні адресні простори.
- Зрозуміло, що стільки фізичної пам'яті, щоб кожному процесу виділити 4Гб, зазвичай немає, та й непотрібно. Реальна, **фізична** пам'ять підставляється ОС в міру потреби.
- ◇ Важливо, що для процесу всі ці процеси підставлення (трансляції адрес) прозорі.
- ◇ Цим займається підсистема віртуальної пам'яті.

<sup>97</sup> Спростуючи, процес — це програма, що виконується, разом зі всією інформацією, яка описує її в нутрошах ОС.

## 32-бітовий адресний простір



- Кожен процес буде мати свій адресний простір, але системні області – простір ядра, часто відображаються в адресний простір всіх процесів.
- Ремарка: якщо Windows передати при завантаженні ключ \3G, його розподіл простору між ядром і користувацьким адресним простором буде як у Linux, 1Gb/3Gb.
- Детальніше див., наприклад, <http://duartes.org/gustavo/blog/post/anatomy-of-a-program-in-memory>

# 64-бітовий адресний простір

## Physical address -- 48 bits

|                                                 |                                          |
|-------------------------------------------------|------------------------------------------|
| Canonical higher half<br>Kernel space<br>127 Tb | 0xFFFF FFFF FFFF FFFF = $2^{64}-1$       |
| Non-canonical addresses<br>Should not be used   | 0x FFFF 8000 0000 0000 = $2^{64}-2^{47}$ |
|                                                 | 0x0000 8000 0000 0000 = $2^{47}$         |
| Canonical lower half<br>User space<br>127 Tb    | 0x0000 0000 0000 0000                    |

- Оскільки фізичних біт адреси у сучасних реалізаціях менше, ніж логічних, розробники процесорів вирішили, що коректними адресами є лише канонічні адреси: від 0 до  $2^{N-1} - 1$  та від  $2^{64} - 2^{N-1}$  до  $2^{64} - 1$ , де  $N$  – кількість фізичних біт.
- Це потрібно для сумісності із майбутніми процесорами – із більшою кількістю фізичних біт адреси.
- Канонічні адреси – ті, де всі біти, старші за  $N$ , або всі є 0 або всі є 1.
- Якщо вважати адреси знаковими числами, враховуючи представлення від'ємних чисел в доповнювальному коді, можна вважати, що це адреси від 0 до  $2^{N-1} - 1$  та від  $-2^{N-1} - 1$  до  $-1$ .

```

55ad540a3000-55ad540d2000 r--p 00000000 08:00 13912 /usr/bin/bash
55ad540d2000-55ad541b1000 r-xp 0002f000 08:00 13912 /usr/bin/bash
55ad541b1000-55ad541eb000 r--p 0010e000 08:00 13912 /usr/bin/bash
55ad541ec000-55ad541f0000 r--p 00148000 08:00 13912 /usr/bin/bash
55ad541f0000-55ad541f9000 rw-p 0014c000 08:00 13912 /usr/bin/bash
55ad541f9000-55ad54204000 rw-p 00000000 00:00 0
55ad556e0000-55ad55904000 rw-p 00000000 00:00 0
[heap]
7fde0ccc0000-7fde0cd26000 r--p 00000000 08:00 3690 /usr/lib/locale/C.utf8/LC_CTYPE
7fde0cd26000-7fde0d00f000 r--p 00000000 08:00 13032 /usr/lib/locale/locale-archive
7fde0d00f000-7fde0d012000 rw-p 00000000 00:00 0
7fde0d012000-7fde0d03a000 r--p 00000000 08:00 11638 /usr/lib/x86_64-linux-gnu/libc.so.6
7fde0d03a000-7fde0d1cf000 r-xp 00028000 08:00 11638 /usr/lib/x86_64-linux-gnu/libc.so.6
7fde0d1cf000-7fde0d227000 r--p 001bd000 08:00 11638 /usr/lib/x86_64-linux-gnu/libc.so.6
7fde0d227000-7fde0d22b000 r--p 00214000 08:00 11638 /usr/lib/x86_64-linux-gnu/libc.so.6
7fde0d22b000-7fde0d22d000 rw-p 00218000 08:00 11638 /usr/lib/x86_64-linux-gnu/libc.so.6
7fde0d22d000-7fde0d23a000 rw-p 00000000 00:00 0
7fde0d23a000-7fde0d248000 r--p 00000000 08:00 27487 /usr/lib/x86_64-linux-gnu/libtinfo.so.6.3
7fde0d248000-7fde0d259000 r-xp 0000e000 08:00 27487 /usr/lib/x86_64-linux-gnu/libtinfo.so.6.3
7fde0d259000-7fde0d267000 r--p 0001f000 08:00 27487 /usr/lib/x86_64-linux-gnu/libtinfo.so.6.3
7fde0d267000-7fde0d26b000 r--p 0002c000 08:00 27487 /usr/lib/x86_64-linux-gnu/libtinfo.so.6.3
7fde0d26b000-7fde0d26c000 rw-p 00030000 08:00 27487 /usr/lib/x86_64-linux-gnu/libtinfo.so.6.3
7fde0d26c000-7fde0d26a000 r--p 00000000 08:00 4455 /usr/lib/locale/C.utf8/LC_NUMERIC
7fde0d26a000-7fde0d26f000 r--p 00000000 08:00 7375 /usr/lib/locale/C.utf8/LC_TIME
.....
7fde0d275000-7fde0d276000 r--p 00000000 08:00 6102 /usr/lib/locale/C.utf8/LC_TELEPHONE
7fde0d276000-7fde0d277000 r--p 00000000 08:00 3784 /usr/lib/locale/C.utf8/LC_MEASUREMENT
7fde0d277000-7fde0d27e000 r--s 00000000 08:00 29849 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7fde0d27e000-7fde0d280000 rw-p 00000000 00:00 0
7fde0d280000-7fde0d282000 r--p 00000000 08:00 3630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fde0d282000-7fde0d2ac000 r-xp 00002000 08:00 3630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fde0d2ac000-7fde0d2b7000 r--p 0002c000 08:00 3630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fde0d2b7000-7fde0d2b8000 r--p 00000000 08:00 3749 /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7fde0d2b8000-7fde0d2ba000 r--p 00037000 08:00 3630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fde0d2ba000-7fde0d2bc000 rw-p 00039000 08:00 3630 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffff51ce3000-7ffff51d04000 rw-p 00000000 00:00 0
7ffff51d25000-7ffff51d29000 r--p 00000000 00:00 0
7ffff51d29000-7ffff51d2a000 r-xp 00000000 00:00 0
[stack]
[.var]
[.vdso]

```

- Отримано із спеціального файлу `/proc/<pid>/maps`.
- Перший стовпчик – діапазон адрес, далі права доступу (р – “приватне відображення”).
- Решта полів пов’язані із відображенням файлів<sup>98</sup>, містять нуль, якщо блок не є відображенням файлу: зміщення у файлі, ід пристрою з якого файл, іноді файлу та його ім’я.
- Стек та купа помічені окремо.

<sup>98</sup> Детальніше – на курсі ОС.

```

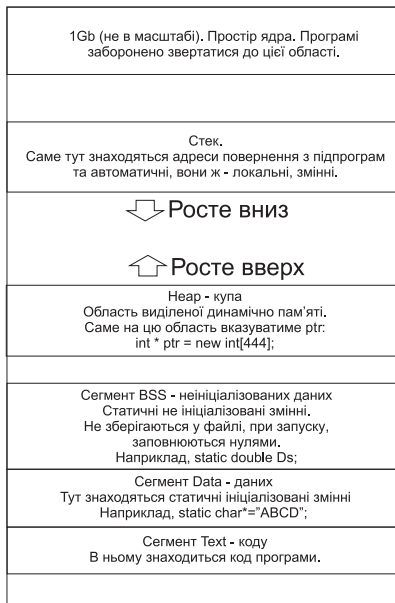
000055ad540a3000 188K r---- bash
000055ad540d2000 892K r-x-- bash
000055ad541b1000 232K r---- bash
000055ad541ec000 16K r---- bash
000055ad541f0000 36K rw--- bash
000055ad541f9000 44K rw--- [anon]
000055ad556e0000 2192K rw--- [anon]
00007fde0ccc000 348K r---- LC_CTYPE
00007fde0cd26000 2980K r---- locale-archive
00007fde0d0f000 12K rw--- [anon]
00007fde0d012000 160K r---- libc.so.6
00007fde0d03a000 1620K r-x-- libc.so.6
00007fde0d1cf000 352K r---- libc.so.6
00007fde0d227000 16K r---- libc.so.6
00007fde0d22b000 8K rw--- libc.so.6
00007fde0d22d000 52K rw--- [anon]
00007fde0d23a000 56K r---- libtinfo.so.6.3
00007fde0d248000 68K r-x-- libtinfo.so.6.3
00007fde0d259000 56K r---- libtinfo.so.6.3
00007fde0d267000 16K r---- libtinfo.so.6.3
00007fde0d26b000 4K rw--- libtinfo.so.6.3
00007fde0d26d000 4K r---- LC_NUMERIC
00007fde0d26e000 4K r---- LC_TIME
.....
00007fde0d275000 4K r---- LC_TELEPHONE
00007fde0d276000 4K r---- LC_MEASUREMENT
00007fde0d277000 28K r--s- gconv-modules.cache
00007fde0d27e000 8K rw--- [anon]
00007fde0d280000 8K r---- ld-linux-x86-64.so.2
00007fde0d282000 168K r-x-- ld-linux-x86-64.so.2
00007fde0d2ac000 44K r---- ld-linux-x86-64.so.2
00007fde0d2b7000 4K r---- LC_IDENTIFICATION
00007fde0d2b8000 8K r---- ld-linux-x86-64.so.2
00007fde0d2ba000 8K rw--- ld-linux-x86-64.so.2
00007fff51ce3000 132K rw--- [stack]
00007fff51d25000 16K r---- [anon]
00007fff51d29000 4K r-x-- [anon]
total 9816K

```

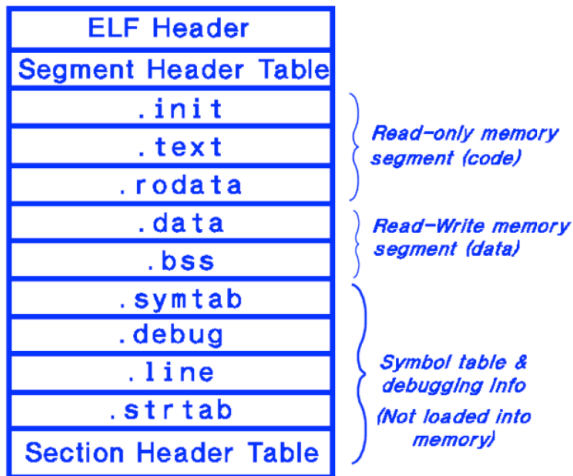
- Інший спосіб отримати цю інформацію – утиліта Linux `pmap`<sup>99</sup>.
- Перший стовпчик – лінійна адреса початку області, другий – її розмір, третій — права доступу: **R**ead, **W**rite, **E**xecute.
- Четверта — якому модулю належить пам'ять.

<sup>99</sup><https://man7.org/linux/man-pages/man1/pmap.1.html>

# Пам'ять процесу детальніше (на прикладі Linux)



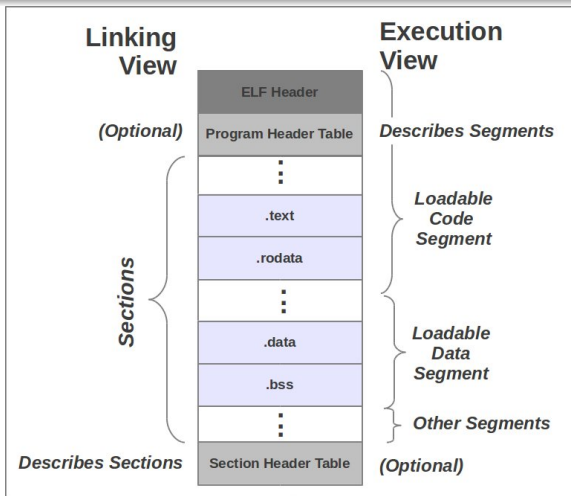
# Будова виконавчих (executable) файлів Linux



Див. також:

<https://www.ics.uci.edu/~aburtsev/238P/hw/hw3-elf/hw3-elf.html>.

## Сегменти та секції



- Секції – сутності, із якими працює лінкер.
- Сегменти<sup>100</sup> – те, з чим працює операційна система, завантажуючи процес.

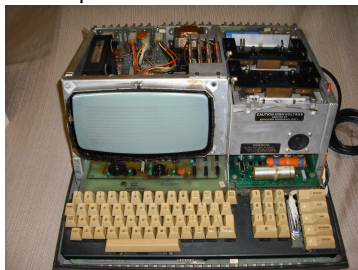
<sup>100</sup>Не слід плутати із фізичними сегментами, що колись використовувалися для керування віртуальною пам'яттю – назва походить від них, але технічно зараз це лише набір сторінок.

- Вправа: зобразіть розташування локальних, глобальних та рівня просторів імен, динамічних, локальних та локальних статичних змінних в пам'яті.

- <https://www.cppstories.com/2023/storage-init-cpp/> – часи існування змінних (automatic, static, thread, dynamic) та види зв'язування (external linkage, module linkage, internal linkage, no linkage, language linkage).



- 8008 був інтегрованою реалізацією центрального процесора "програмованого термінала" CDC Datapoint 2200<sup>101</sup>:



- Перше фото з [https://en.wikipedia.org/wiki/Datapoint\\_2200](https://en.wikipedia.org/wiki/Datapoint_2200), друге – з <http://bugbookmuseum.blogspot.com/2015/10/8008-microprocessor-and-how-it-became.html>.

<sup>101</sup>В той час, щоб уникати конфліктів із виробниками "справжніх", великих комп'ютерів – мейнфреймів, "Big iron", те, що ми зараз назвали б персональним комп'ютером, називали терміналами, засобами розробки, текстовими процесорами тощо, лише – не б комп'ютерами.

## Ранні мікропроцесори Intel та DataPoint 2200 – посилання

- <http://www.righto.com/2015/05/the-texas-instruments-tmx-1795-first.html> – Texas Instruments TMX 1795, клон 8008, створений із тією ж ціллю – замінити Datapoint 2200, судячи по всьому – за документацією Інтел, і, ймовірно – ніколи не працював повноцінно. З іншого боку, може це пропаганда Інтел. Стаття містить посилання на цілу серію інтерв'ю "Oral history" від Computer History Museum.
- <http://www.rogerarrick.com/osiris/burgett.txt> – Інтерв'ю з програмістом Intel, який розробляв інструменти розробки під 4004-8008-8080 тощо.
- <https://www.youtube.com/watch?v=DrJiYysZwxk> – відео-огляд Datapoint 2200.
- <http://www.os2museum.com/wp/the-ibm-pc-bios-and-intel-isis-ii/> – ОС від Інтел, для системи розробки під 8080.
- <http://oldcomputers.net/datapoint-2200.html> – фото.
- <http://www.home.neab.net/gandalf/ClassicComputing/Pictures/DP-2200/> – фото.

# Історія переривань

- Слід зазначити, що ідея (асинхронних) переривань – очевидна та всезагально поширена зараз, тривалий час викликала суперечки. Популярність прийшла до неї далеко не зразу.
- Гарний огляд історії переривань:  
<https://people.computing.clemson.edu/~mark/interrupts.html>
  - ◇ Вперше – UNIVAC 1103 / 1103A, 1953 або 1956.
  - ◇ Перші переривання вводу-виводу: NBS DYSEAC (1954).
  - ◇ Перший вектор переривань – Lincoln Labs TX-2, 1957.
  - ◇ DMA – ймовірно, IBM SAGE (1952) – дискусійно, NBS DYSEAC (1954).
  - ◇ Розділ "CDC 6600 (1964)" та "Turner and Rawlings' comments on interrupts" – щодо дискусій за і проти.
  - ◇ Зокрема, асинхронні переривання утруднюють не тільки пошук помилок, але вирізнення – помилка програмна, чи апаратна, оскільки код може успішно працювати до певного збігу умов та обставин.
  - ◇ Xerox Mesa Processor (1981): "An interrupt caused a specified condition variable<sup>102</sup> to be notified and thus wake up a waiting process"
- Щодо термінології – оці всі exception, trap, syscall, external interrupt тощо: див. розділ "Design Tree for Interrupts".

<sup>102</sup>Що це – будемо вивчати на АКС.

- Krste Asanović is one of the chief architects of the RISC-V instruction set architecture. He described two major system design viewpoints regarding interrupts, each of which is supported in RISC-V:
  - High-performance, Unix-like systems:
    - ◇ Interrupts are treated as data.
    - ◇ The interrupt handlers do minimal work, and most of the work done in response to interrupts is done sometime later.
    - ◇ Task scheduling is primarily performed by the OS scheduler.
  - Embedded systems :
    - ◇ Interrupts are treated as control.
    - ◇ Interrupt handling is a significant fraction of the workload, and a significant fraction of code is structured as interrupt handlers.
    - ◇ Task scheduling is primarily performed by the interrupt controller.
  - High-performance real-time systems:
    - ◇ Can't waste time on interrupt overhead
    - ◇ Handlers poll I/O devices with regular heartbeat
  - Презентація: [https://riscv.org/wp-content/uploads/2016/07/Tue0900\\_RISCV-20160712-InterruptsV2.pdf](https://riscv.org/wp-content/uploads/2016/07/Tue0900_RISCV-20160712-InterruptsV2.pdf), виступ: [https://www.youtube.com/watch?v=iPbaG\\_wnNJY](https://www.youtube.com/watch?v=iPbaG_wnNJY).

Цитата із казки "Чарівна круговерть" Євгена Наумова, яка гарно описує світанок комп'ютерного світу:

- Було це чи не було, казка це чи бувальщина, тільки бачив я все на власні очі, - почав Дід Драндулет. - Коли з'явилися перші автомобілі, вони самі ще не знали, якими їм бути. І небачені чудовиська запрудили шляхи.

Рогаті автомашини. Їжаки-автомобілі, в яких кузова були всіяні гострими сталевими голками. Гнучкі й довгі машини-гадюки. Круглі та тугі, ніби м'яч, грузовики. Вони могли підібгати колеса й котитися бездоріжжям. Автострибуни.

Були навіть автомобілі, які могли розсіпатися на безліч маленьких автомобільчиків, пролізти через будь-яку шпаринку, а потім з'єднатися і знову їхати далі.

Були машини з вісьмома колесами - чотири внизу, стільки ж зверху. Коли вони перевертались, то могли спокійнісінько їхати далі догори дном.

Зустрічалися триколісні та двоколісні. А одного разу з'явився автомобіль на одному колесі! Чудернацький вигляд він мав - довгий і тонкий, мов телеграфний стовп.

- Як же він їздив на одному колесі? - здивувався Малько-Ванько.

- Ще й як їздив! Автостоп - так його стали називати - хвацько носився не тільки по дорогах, а й по стежках, бордюрах і навіть... по карнизах будинків.

- А по карнизах навіщо? Хіба була потреба?

- Потреба була! Тому що - і це було найголовнішим - автомобілі не тільки не знали, який вигляд вони мали, а й не відали, як їздити. Кожен мчав, куди хотів і як хотів.

Плутанина, безладдя! І була це країна Безладна Круговерть. На дорогах весь час траплялися сутички, аварії, чвари, скандали, бійки. То їжак штрикне когось своїми колючками, то Рогач потовче, то Автогадюка задушить у сталевих обіймах... На дорогах затори, пробки! А машин ставало дедалі більше.

## Apollo 11 Guidance Computer (AGC) vs USB-C Chargers

I think it is healthy to compare historical and modern computing. Let's see how the CPUs contained in recent USB-C wall chargers compare to the power of the Apollo 11 Guidance Computer (AGC). The Apollo 11 spacecraft carried 3 humans to the moon and back in 1969.

| Device                | Google Pixel 18W Charger                               | Huawei 40W SuperCharge                                | Anker PowerPort Atom PD 2                               | Apollo 11 Moon Landing Guidance Computer (AGC)                                                                                                                                             |
|-----------------------|--------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function              | Charges a phone                                        | Charges a phone or maybe a laptop                     | Charges 2 phones or maybe laptops                       | <ul style="list-style-type: none"> <li>• Fly most-of-the-way to moon (CSM)</li> <li>• Land on moon (LEM)</li> <li>• Take off from moon (LEM)</li> <li>• Fly back to Earth (CSM)</li> </ul> |
| Microchip(s)          | Weltrend WT6630P                                       | Richtek RT7205                                        | Cypress CYPD4225                                        | Discrete components                                                                                                                                                                        |
| Clock Speed           | 10 MHz                                                 | 22.7 MHz                                              | 48 MHz                                                  | 1.024 MHz                                                                                                                                                                                  |
| RAM                   | 512 bytes                                              | "0.75kB"                                              | 8KB                                                     | 2048 15-bit words / 4KB if you include the parity bit in each word                                                                                                                         |
| Program Storage Space | 8KB                                                    | 24KB (Mask ROM + OTP)                                 | 128KB Flash                                             | 36,864 15-bit words / 72KB if you include the parity bit in each word                                                                                                                      |
| Instruction Set       | Intel 8051 (8-bit)                                     | Unknown                                               | ARM Cortex-M0 32-bit implementing ARMv6-M               | 16-bit accumulator based                                                                                                                                                                   |
| Sources               | <a href="#">ChargerLabs Teardown WT6630P Datasheet</a> | <a href="#">ChargerLabs Teardown RT7205 Datasheet</a> | <a href="#">ChargerLabs Teardown CYPD4225 Datasheet</a> | <a href="#">CPU description</a> <a href="#">Memory Functional overview</a>                                                                                                                 |

- Цитата з [The ENIAC](#), виділення моє: *"ENIAC was capable of around five hundred FLOPS depending upon the work load. While this seems very slow today, it was still roughly a thousand times faster than any other machine at that time. In the early days of its service, **the machine worked roughly half of any given day**, and these breakdowns were almost always due to vacuum tube failures. This was followed by improvements in components around 1948 which increased the mean time to failure to about two days. Despite what seems to modern minds to be an unacceptable failure rate, ENIAC was better than the machines before it, and better than many machines that immediately followed."*

## Посилання

- ["An All-Optical General-Purpose CPU and Optical Computer Architecture"](#), 2024. Включає практичну реалізацію.
- ["Some useful tools for binary formats"](#) – список утиліт для роботи з бінарними файлами.
- ◇ Зокрема: [Kaitai Struct](#) – бінарний формат описується спеціальними скриптами. Містить багато прикладів, зокрема – форматів виконавчих файлів, VFAT, GPT тощо. Генерує код для C++, Java, Lua, Go і багатьох інших мов. Поки read only.
- Про баги, пов'язані з особливостями заліза: ["Better Reboot Your Boeing 787 Every Three Weeks"](#), ["AMD's EPYC Rome Chips Crash After 1,044 Days of Uptime"](#), ["LightSail \[1\] Solar Sail Test Flight Stalled by Software Glitch"](#) – переповнився лог телеметрії на флеш-картці, але відновили – і він потім успішно відпрацював.
- ["An Introduction to Low-Latency Scripting for Game Engines"](#) – віртуальна машина RISC-V для скриптингу ігор.

## Посилання

- Серія статей про декомпіляцію: "30 Years of Decompilation and the Unsolved Structuring Problem: Part 1", "Part 2", "Decompiling 2024: A Year of Resurgence in Decompilation Research".
- "Pointer Authentication" – огляд підтримки підписування вказівників апаратним забезпеченням та в Clang.
- "Embedded Systems Roadmap: Bridging the Gap", репозиторій.
- "Wuffs – Wrangling Untrusted File Formats Safely", цікава спеціалізована мова<sup>103</sup> для більш безпечного читання файлів.
- "CARDIAC" – вивчення ISA/асемблера, використовуючи паперовий комп'ютер.

---

<sup>103</sup>DSP – Domain specific language.

